

Team Notebook

Ateneo de Manila University

The 100 Last Minute Bugs That Really, Really, Really, Really, Really Love You

June 16, 2026

Contents

1	Boilerplate	3
1.1	Template	3
1.2	Debug	3
1.3	Generator-Checker	3
2	Data Structures	3
2.1	Sparse Table	3
2.2	Square Root Decomposition	4
2.3	Segment Tree Monoids	4
2.4	Segment Tree	5
2.5	2D Segment Tree	5
2.6	Dynamic Segment Tree	6
2.7	Persistent Segment Tree	7
2.8	Kth Smallest Persistent Segment Tree	8
2.9	Segment Tree Beats	8
3	Geometry	9
3.1	Point	9
3.2	Area of a Polygon	9
3.3	Intersection	9
3.4	Get Length of Segment Union	10
3.5	Get Intersecting Segments	10
3.6	Get Minimum Enclosing Circle	10
3.7	Get Nearest Pair of Points	10
3.8	Li-Chao Tree	10
4	Graphs	11
4.1	Topological Sort	11
4.2	Cycle Find DFS	11
4.3	Disjoint Set Union/UnionFind	11
4.4	Shortest Paths	11
4.4.1	Dijkstra	11
4.4.2	Bellman-Ford	12
4.4.3	Floyd-Warshall	12
4.5	Trees	12
4.5.1	Lowest Common Ancestor	12
4.5.2	Heavy Light Decomposition	12
4.5.3	Centroid Decomposition	12
4.6	Connectivity and Components	13
4.6.1	Bridge-Finding Algorithm	13
4.6.2	Block-Cut Tree	13

4.6.3	Cutpoints-Bridges	13
4.6.4	Kosaraju	13
4.6.5	2SAT	13
4.6.6	Flows and Bipartite Matching	14
4.6.7	Dinic	14
4.6.8	Ford-Fulkerson Directed	14
4.6.9	Ford-Fulkerson Undirected	14
4.6.10	Kuhn	15
4.6.11	Min Cost Max Flow	15

5	Math	15
5.1	Binary and Ternary Search	15
5.2	Modulo Integer	16
5.3	Matrix	17
5.4	Modulo Pow	20
5.5	Modulo Combinatorics	20
5.6	Dynamic MEX	20
5.7	Extended Euclidean GCD	20
5.8	Sieve of Eratosthenes	20
5.9	Linear Sieve	20
5.10	Segmented Sieve	21
5.11	Is Prime Miller-Rabin	21
5.12	Get One Factor Brent's Pollard-Rho	21
5.13	Get Prime Factors	21
5.14	Get All Factors	21
5.15	Euler's Totient Function	22
5.16	Number of Divisors	22
5.17	Sum of Divisors	22
5.18	Inverse Modulo	22
5.19	Chinese Remainder Theorem	22
5.20	Garner	22
5.21	Modulo Log	22
5.22	Primitive Root	22
5.23	Modulo Root	23
5.24	Floor Sum	23
5.25	Montgomery Multiplication	23
5.26	Poly FFT	23
5.27	Moly NTT	24
5.28	Fast Is Square	26
5.29	Josephus Problem	26
5.30	Simpson Integration	26
5.31	Solve Diophantine Equation	26
5.32	Solve Modulo Equation	26
6	Misc	27
6.1	Custom Hash	27
6.2	Fast Convolutions	27
6.3	Kadane's Algorithm	27
6.4	Longest Common Subsequence	27
6.5	Longest Common Substring	28
6.6	Longest Increasing Subsequence	28
6.7	Random	28
6.8	Z-Algorithm	28

7	Formulas	28
7.1	Physics	28
7.2	Markov Chains	28
7.3	Burnside's Lemma	28
7.4	Bézout's identity	28
7.5	Misc	29
7.5.1	Determinants and PM	29
7.5.2	BEST Theorem	29
7.5.3	Primitive Roots	29
7.5.4	Sum of primes	29
7.5.5	Floor	29
7.5.6	Large Primes	29
7.6	Other Combinatorics Stuff	30
7.7	The Twelfefold Way	30

```
// S:  $O(n * \log(n))$ , U: NA, Q:  $O(1)$  or  $O(\log(n))$ , M:  $O(n * \log(n))$ 
template<typename T, typename F>
struct SparseTable {
    int n;
```

```

static constexpr inline S map(F f, const S &a) {
    if (a.len == 0) { return a; }
    S res;
    res.And = ((a.And & f.a) & ~f.b) | (~a.Or & f.a) & f.b;
    res.Or = ((a.Or & f.a) & ~f.b) | (~a.And & f.a) & f.b;
    res.Xor = (a.Xor & f.a) ^ (a.len & 1 ? f.b : 0);
    res.len = a.len;
    return res;
}

static constexpr inline F cmp(F f, F g) { return {g.a & f.a, (g.b &
f.a) ^ f.b}; }

/*
void andUpdate(lng l, lng r, lng x) { update(l, r, {x, 0}); }
void orUpdate(lng l, lng r, lng x) { update(l, r, {x, x}); }
void xorUpdate(lng l, lng r, lng x) { update(l, r, {~0LL, x}); }
void setUpdate(lng l, lng r, lng x) { update(l, r, {0, x}); }
*/
};

template<bool opt = false>
struct MonGcd {
    struct S { lng Gcd, Lcm, len; };
    struct F {
        lng a, b;
        bool operator==(const F&) const = default;
    };

    static constexpr inline S idS() {
        if constexpr (opt) { return {0, 1, 0}; }
        return {0, 0, 0};
    }

    static constexpr inline F idF() { return {0, 1}; }
    static constexpr inline S defR(lng l, lng r) {
        if constexpr (opt) { return {0, 1, r - 1 + 1}; }
        return {0, 0, r - 1 + 1};
    }

    static constexpr inline S init(lng i, lng x) {
        if constexpr (opt) { return {x, x, 1}; }
        return {x, 0, 1};
    }

    static constexpr inline S ope(const S &a, const S &b) {
        if (a.len == 0) { return b; }
        if (b.len == 0) { return a; }
        S res;
        res.Gcd = gcd(a.Gcd, b.Gcd);
        if constexpr (opt) { res.Lcm = lcm(a.Lcm, b.Lcm); }
        res.len = a.len + b.len;
        return res;
    }

    static constexpr inline S map(F f, const S &a) {
        if (a.len == 0) { return a; }
        S res;
        res.Gcd = lcm(gcd(a.Gcd, f.a), f.b);
        if constexpr (opt) { res.Lcm = lcm(gcd(a.Lcm, f.a), f.b); }
        res.len = a.len;
        return res;
    }

    static constexpr inline F cmp(F f, F g) { return {gcd(g.a, f.a),
lcm(gcd(g.b, f.a), f.b)}; }

/*
void gcdUpdate(lng l, lng r, lng x) { update(l, r, {x, 1}); }
void lcmUpdate(lng l, lng r, lng x) { update(l, r, {0, x}); }
void setUpdate(lng l, lng r, lng x) { update(l, r, {1, x}); }
*/
};

struct MonSar {
    struct S {
        lng sum, max_prf, max_suf, max_sum, min_prf, min_suf, min_sum,
len;

```

```

    void rev() { std::swap(max_prf, max_suf); std::swap(min_prf, min_suf); }
};
struct F {
    bool to_set; lng set_upd;
    bool operator==(const F&) const = default;
};

static constexpr inline S idS() { return {0, -INF64, -INF64, -INF64, INF64, INF64, INF64, 0}; }
static constexpr inline F idF() { return {false, 0}; }
static constexpr inline S defR(lng l, lng r) { return {0, 0, 0, 0, 0, 0, 0, r - l + 1}; }
static constexpr inline S init(lng i, lng x) { return {x, x, x, x, x, x, x, 1}; }

static constexpr inline S ope(const S &a, const S &b) {
    if (a.len == 0) { return b; }
    if (b.len == 0) { return a; }
    S res;
    res.sum = a.sum + b.sum;
    res.max_prf = max(a.max_prf, a.sum + b.max_prf);
    res.max_suf = max(b.max_suf, b.sum + a.max_suf);
    res.max_sum = max({a.max_sum, b.max_sum, a.max_suf + b.max_prf});
    res.min_prf = min(a.min_prf, a.sum + b.min_prf);
    res.min_suf = min(b.min_suf, b.sum + a.min_suf);
    res.min_sum = min({a.min_sum, b.min_sum, a.min_suf + b.min_prf});
    res.len = a.len + b.len;
    return res;
}

static constexpr inline S map(F f, const S &a) {
    if (!f.to_set || a.len == 0) { return a; }
    S res = a;
    res.sum = f.set_upd * a.len;
    res.max_prf = res.max_suf = res.max_sum = f.set_upd > 0 ? res.sum : f.set_upd;
    res.min_prf = res.min_suf = res.min_sum = f.set_upd < 0 ? res.sum : f.set_upd;
    return res;
}

static constexpr inline F cmp(F f, F g) { return f.to_set ? f : g; }
}

struct MonSet {
    struct S { /* states */ lng len; };
    struct F {
        bool to_set; lng set_upd;
        bool operator==(const F&) const = default;
    };

    static constexpr inline S idS() { return {0}; }
    static constexpr inline F idF() { return {false, 0}; }
    static constexpr inline S defR(lng l, lng r) { return {r - l + 1}; }
    static constexpr inline S init(lng i, lng x) { return {1}; }

    static constexpr inline S ope(const S &a, const S &b) {
        if (a.len == 0) { return b; }
        if (b.len == 0) { return a; }
        S res;
        // ope states
        res.len = a.len + b.len;
        return res;
    }

    static constexpr inline S map(F f, const S &a) {
        if (!f.to_set || a.len == 0) { return a; }
        S res = a;
        // map states
        return res;
    }
}

```

```

static constexpr inline F cmp(F f, F g) { return f.to_set ? f : g; }
}

2.4 Segment Tree

// S: O(n), U: O(log(n)), Q: O(log(n)), M: O(n)
template<typename Mon>
struct SegTree {
    using S = typename Mon::S;
    using F = typename Mon::F;

    int n;
    vector<S> tree;
    vector<F> lazy;

    template<typename T>
    SegTree(std::span<const T> &a) : n(a.size()),
        tree(2 * std::bit_ceil(static_cast<unsigned>(max(n, 1))),
            Mon::idS()),
        lazy(2 * std::bit_ceil(static_cast<unsigned>(max(n, 1))),
            Mon::idF()) {
        if (n > 0) { build(1, 0, n - 1, a); }
    }

    template<typename T>
    void build(int i, int l, int r, std::span<const T> &a) {
        if (l == r) { tree[i] = Mon::init(l, a[l]); return; }
        int md = std::midpoint(l, r);
        build(2 * i, l, md, a);
        build(2 * i + 1, md + 1, r, a);
        tree[i] = Mon::ope(tree[2 * i], tree[2 * i + 1]);
    }

    inline void apply(int i, F f) {
        tree[i] = Mon::map(f, tree[i]);
        lazy[i] = Mon::cmp(f, lazy[i]);
    }

    inline void push(int i) {
        if (lazy[i] == Mon::idF()) { return; }
        apply(2 * i, lazy[i]);
        apply(2 * i + 1, lazy[i]);
        lazy[i] = Mon::idF();
    }

    void update(int i, int l, int r, int ql, int qr, F f) {
        if (ql <= l && r <= qr) { apply(i, f); return; }
        push(i);
        int md = std::midpoint(l, r);
        if (ql <= md) { _update(2 * i, l, md, ql, qr, f); }
        if (qr > md) { _update(2 * i + 1, md + 1, r, ql, qr, f); }
        tree[i] = Mon::ope(tree[2 * i], tree[2 * i + 1]);
    }

    void update(int l, int r, F f) {
        l = max(l, 0); r = min(r, n - 1);
        if (l <= r) { _update(1, 0, n - 1, l, r, f); }
    }

    template<class G>
    int _maxR(int i, int l, int r, int ql, G &g, S &acc) {
        if (r < ql) { return -1; }
        if (ql <= l) {
            S nac = Mon::ope(acc, tree[i]);
            if (g(nac)) { acc = nac; return -1; }
        }
        if (l == r) { return l - 1; }
        push(i);
        int md = std::midpoint(l, r);
        int res = _maxR(2 * i, l, md, ql, g, acc);
        if (res != -1) { return res; }
        return _maxR(2 * i + 1, md + 1, r, ql, g, acc);
    }

    template<class G>
    int maxR(int l, G g) { // Returns max r in [l, n - 1] where
        g(query(l, r)) is true.
    }
}

```

```

    l = max(l, 0);
    if (l > n - 1) { return l - 1; }
    S acc = Mon::idS();
    int res = _maxR(1, 0, n - 1, l, g, acc);
    return res == -1 ? n - 1 : res;
}

template<class G>
int _minL(int i, int l, int r, int qr, G &g, S &acc) {
    if (l > qr) { return -1; }
    if (r <= qr) {
        S nac = Mon::ope(tree[i], acc);
        if (g(nac)) { acc = nac; return -1; }
    }
    if (l == r) { return l + 1; }
    push(i);
    int md = std::midpoint(l, r);
    int res = _minL(2 * i + 1, md + 1, r, qr, g, acc);
    if (res != -1) { return res; }
    return _minL(2 * i, l, md, qr, g, acc);
}

template<class G>
int minL(int r, G g) { // Returns min l in [0, r] where g(query(l,
    r)) is true.
    r = min(r, n - 1);
    if (r < 0) { return 0; }
    S acc = Mon::idS();
    int res = _minL(1, 0, n - 1, r, g, acc);
    return res == -1 ? 0 : res;
}

S_query(int i, int l, int r, int ql, int qr) {
    if (ql <= l && r <= qr) { return tree[i]; }
    push(i);
    int md = std::midpoint(l, r);
    if (qr <= md) { return _query(2 * i, l, md, ql, qr); }
    if (ql > md) { return _query(2 * i + 1, md + 1, r, ql, qr); }
    return Mon::ope(_query(2 * i, l, md, ql, qr), _query(2 * i + 1,
        md + 1, r, ql, qr));
}

S_query(int l, int r) {
    l = max(l, 0); r = min(r, n - 1);
    if (l > r) { return Mon::idS(); }
    return _query(1, 0, n - 1, l, r);
}

S_queryAll() { return tree[1]; }
}

```

2.5 2D Segment Tree

```

// S: O(n^2), U: O(log(n)^2), Q: O(log(n)^2), M: O(n^2)
template<typename Mon>
struct SegTree2D {
    using S = typename Mon::S;
    using F = typename Mon::F;

    int n, m;
    vector<vector<S>> tree;

    template<typename T>
    SegTree2D(const vector<vector<T>> &a) : n(a.size()), m(a.empty() ?
        0 : a[0].size()) {
        if (n == 0 || m == 0) { return; }
        tree.assign(4 * n, vector<S>(4 * m, Mon::idS()));
        build(1, 0, n - 1, a);
    }

    template<typename T>
    void _build(int i, int l1, int r1, int j, int l2, int r2, const
        vector<vector<T>> &a) {
        if (l2 == r2) {
            if (l1 == r1) { tree[i][j] = Mon::init(l2, a[l1][l2]); }
            else { tree[i][j] = Mon::ope(tree[2 * i][j],
                tree[2 * i + 1][j]); }
        }
    }
}

```

```

    return;
}
int md = 12 + (r2 - 12) / 2;
_build(i, l1, r1, 2 * j, 12, md, a);
_build(i, l1, r1, 2 * j + 1, md + 1, r2, a);
tree[i][j] = Mon::ope(tree[i][2 * j], tree[i][2 * j + 1]);
}
template<typename T>
void build(int i, int l1, int r1, const vector<vector<T>> &a) {
    if (l1 != r1) {
        int md = l1 + (r1 - l1) / 2;
        build(2 * i, l1, md, a);
        build(2 * i + 1, md + 1, r1, a);
    }
    _build(i, l1, r1, 1, 0, m - 1, a);
}

void _update_j(int i, int l1, int r1, int j, int l2, int r2, int qj, F f) {
    if (l2 == r2) {
        if (l1 == r1) { tree[i][j] = Mon::map(f, tree[i][j]); }
        else { tree[i][j] = Mon::ope(tree[2 * i][j], tree[2 * i + 1][j]); }
        return;
    }
    int md = 12 + (r2 - 12) / 2;
    if (qj <= md) { _update_j(i, l1, r1, 2 * j, 12, md, qj, f); }
    else { _update_j(i, l1, r1, 2 * j + 1, md + 1, r2, qj, f); }
    if (l1 == r1) { tree[i][j] = Mon::ope(tree[i][2 * j], tree[i][2 * j + 1]); }
    else { tree[i][j] = Mon::ope(tree[2 * i][j], tree[2 * i + 1][j]); }
}

void _update_i(int i, int l1, int r1, int qi, int qj, F f) {
    if (l1 != r1) {
        int md = l1 + (r1 - l1) / 2;
        if (qi <= md) { _update_i(2 * i, l1, md, qi, qj, f); }
        else { _update_i(2 * i + 1, md + 1, r1, qi, qj, f); }
    }
    _update_j(i, l1, r1, 1, 0, m - 1, qj, f);
}

void update(int i, int j, F f) {
    if (i >= 0 && i < n && j >= 0 && j < m) { _update_i(1, 0, n - 1, i, j, f); }
}

S _query_j(int i, int j, int l2, int r2, int qj1, int qj2) {
    if (qj1 <= 12 && r2 <= qj2) { return tree[i][j]; }
    int md = 12 + (r2 - 12) / 2;
    if (qj2 <= md) { return _query_j(i, 2 * j, 12, md, qj1, qj2); }
    if (qj1 > md) { return _query_j(i, 2 * j + 1, md + 1, r2, qj1, qj2); }
    return Mon::ope(_query_j(i, 2 * j, 12, md, qj1, qj2), _query_j(i, 2 * j + 1, md + 1, r2, qj1, qj2));
}

S _query_i(int i, int l1, int r1, int qi1, int qj1, int qi2, int qj2) {
    if (qi1 <= l1 && r1 <= qi2) { return _query_j(i, 1, 0, m - 1, qj1, qj2); }
    int md = l1 + (r1 - l1) / 2;
    if (qi2 <= md) { return _query_i(2 * i, l1, md, qi1, qj1, qi2, qj2); }
    if (qi1 > md) { return _query_i(2 * i + 1, md + 1, r1, qi1, qj1, qi2, qj2); }
    return Mon::ope(_query_i(2 * i, l1, md, qi1, qj1, qi2, qj2), _query_i(2 * i + 1, md + 1, r1, qi1, qj1, qi2, qj2));
}

```

```

}
S query(int i1, int j1, int i2, int j2) {
    i1 = max(i1, 0); i2 = min(i2, n - 1);
    j1 = max(j1, 0); j2 = min(j2, m - 1);
    if (i1 > i2 || j1 > j2) { return Mon::idS(); }
    return _query_i(1, 0, n - 1, i1, j1, i2, j2);
}

S queryAll() {
    if (n == 0 || m == 0) { return Mon::idS(); }
    return tree[1][1];
}

};

2.6 Dynamic Segment Tree

// S: O(q * log(n)), U: O(log(n)), Q: O(log(n)), M: O(q * log(n))
template<typename Mon>
struct DynSegTree {
    using S = typename Mon::S;
    using F = typename Mon::F;

    struct Node {
        S val; F lzy; int lc = -1, rc = -1;
        Node(lng l, lng r) : val(Mon::defR(l, r)), lzy(Mon::idF()) {}
    };
    lng n;
    int root = -1;
    vector<Node> tree;

    DynSegTree(lng N) : n(N) {
        tree.reserve(500'000);
        if (n > 0) { root = createNode(0, n - 1); }
    }
    template<typename T>
    DynSegTree(std::span<const T> &a) : n(a.size()) {
        tree.reserve(2 * std::bit_ceil(static_cast<unsigned>(max(n, 1LL))));
        if (n > 0) { root = build(0, n - 1, a); }
    }
    template<typename T>
    int build(lng l, lng r, std::span<const T> &a) {
        int cur = createNode(l, r);
        if (l == r) { tree[cur].val = Mon::init(l, a[l]); return cur; }
        lng md = std::midpoint(l, r);
        int nlc = build(l, md, a); tree[cur].lc = nlc;
        int nrc = build(md + 1, r, a); tree[cur].rc = nrc;
        tree[cur].val = Mon::ope(tree[nlc].val, tree[nrc].val);
        return cur;
    }
    int createNode(lng l, lng r) {
        tree.emplace_back(l, r);
        return tree.size() - 1;
    }

    inline void apply(int i, F f) {
        tree[i].val = Mon::map(f, tree[i].val);
        tree[i].lzy = Mon::cmp(f, tree[i].lzy);
    }

    inline void push(int i, lng l, lng r) {
        if (tree[i].lzy == Mon::idF()) { return; }
        lng md = std::midpoint(l, r);
        if (tree[i].lc == -1) { int nlc = createNode(l, md); tree[i].lc = nlc; }
        if (tree[i].rc == -1) { int nrc = createNode(md + 1, r); tree[i].rc = nrc; }
        apply(tree[i].lc, tree[i].lzy);
        apply(tree[i].rc, tree[i].lzy);
        tree[i].lzy = Mon::idF();
    }

    void _update(int i, lng l, lng r, lng ql, lng qr, F f) {
        if (ql <= l && r <= qr) { apply(i, f); return; }
        push(i, l, r);
        lng md = std::midpoint(l, r);

```

```

        if (ql <= md) {
            if (tree[i].lc == -1) { int nlc = createNode(l, md); tree[i].lc = nlc; }
            _update(tree[i].lc, l, md, ql, qr, f);
        }
        if (qr > md) {
            if (tree[i].rc == -1) { int nrc = createNode(md + 1, r); tree[i].rc = nrc; }
            _update(tree[i].rc, md + 1, r, ql, qr, f);
        }
        S l_val = tree[i].lc != -1 ? tree[tree[i].lc].val : Mon::defR(l, md);
        S r_val = tree[i].rc != -1 ? tree[tree[i].rc].val : Mon::defR(md + 1, r);
        tree[i].val = Mon::ope(l_val, r_val);
    }
    void update(lng l, lng r, F f) {
        l = max(l, 0LL); r = min(r, n - 1);
        if (l <= r) { _update(root, 0, n - 1, l, r, f); }
    }

    template<class G>
    lng _maxR(int i, lng l, lng r, lng ql, G &g, S &acc) {
        if (r < ql) { return -1; }
        if (ql <= l) {
            S nac = Mon::ope(acc, i == -1 ? Mon::defR(l, r) : tree[i].val);
            if (g(nac)) { acc = nac; return -1; }
        }
        if (l == r) { return l - 1; }
        if (i != -1) { push(i, l, r); }
        lng md = std::midpoint(l, r);
        lng res = _maxR(i == -1 ? -1 : tree[i].lc, l, md, ql, g, acc);
        if (res != -1) { return res; }
        return _maxR(i == -1 ? -1 : tree[i].rc, md + 1, r, ql, g, acc);
    }
    template<class G>
    lng maxR(lng l, G g) { // Returns max r in [l, n - 1] where g(query(l, r)) is true.
        if (l < 0 || l > n - 1) { return l - 1; }
        S acc = Mon::idS();
        lng res = _maxR(root, 0, n - 1, l, g, acc);
        return res == -1 ? n - 1 : res;
    }

    template<class G>
    lng _minL(int i, lng l, lng r, lng qr, G &g, S &acc) {
        if (l > qr) { return -1; }
        if (r <= qr) {
            S nac = Mon::ope(i == -1 ? Mon::defR(l, r) : tree[i].val, acc);
            if (g(nac)) { acc = nac; return -1; }
        }
        if (l == r) { return l + 1; }
        if (i != -1) { push(i, l, r); }
        lng md = std::midpoint(l, r);
        lng res = _minL(i == -1 ? -1 : tree[i].rc, md + 1, r, qr, g, acc);
        if (res != -1) { return res; }
        return _minL(i == -1 ? -1 : tree[i].lc, l, md, qr, g, acc);
    }
    template<class G>
    lng minL(lng r, G g) { // Returns min l in [0, r] where g(query(l, r)) is true.
        if (r < 0 || r > n - 1) { return r + 1; }
        S acc = Mon::idS();
        lng res = _minL(root, 0, n - 1, r, g, acc);
        return res == -1 ? 0 : res;
    }

    S _query(int i, lng l, lng r, lng ql, lng qr) {
        if (i == -1) {

```

```

lng _maxR(int i, lng l, lng r, lng ql, G &g, S &acc, F tac) {
    if (r < ql) { return -1; }
    if (ql <= l) {
        S val = i == -1 ? Mon::defR(l, r) : tree[i].val; val =
            Mon::map(tac, val);
        S nac = Mon::ope(acc, val);
        if (g(nac)) { acc = nac; return -1; }
    }
    if (l == r) { return l - 1; }
    tac = i == -1 ? tac : Mon::cmp(tac, tree[i].lzy);
    lng md = std::midpoint(l, r);
    lng res = _maxR(i == -1 ? -1 : tree[i].lc, l, md, ql, g, acc,
        tac);
    if (res != -1) { return res; }
    return _maxR(i == -1 ? -1 : tree[i].rc, md + 1, r, ql, g, acc,
        tac);
}

template<class G>
lng maxR(int ver, lng l, G g) { // Returns max r in [l, n - 1]
    where g(query(l, r)) is true.
    if (l < 0 || l > n - 1) { return l - 1; }
    S acc = Mon::idS();
    lng res = _maxR(roots[ver], 0, n - 1, l, g, acc, Mon::idF());
    return res == -1 ? n - 1 : res;
}

template<class G>
lng _minL(int i, lng l, lng r, lng qr, G &g, S &acc, F tac) {
    if (l > qr) { return -1; }
    if (r <= qr) {
        S val = i == -1 ? Mon::defR(l, r) : tree[i].val; val =
            Mon::map(tac, val);
        S nac = Mon::ope(val, acc);
        if (g(nac)) { acc = nac; return -1; }
    }
    if (l == r) { return l + 1; }
    tac = i == -1 ? tac : Mon::cmp(tac, tree[i].lzy);
    lng md = std::midpoint(l, r);
    lng res = _minL(i == -1 ? -1 : tree[i].rc, md + 1, r, qr, g,
        acc, tac);
    if (res != -1) { return res; }
    return _minL(i == -1 ? -1 : tree[i].lc, l, md, qr, g, acc,
        tac);
}

template<class G>
lng minL(int ver, lng r, G g) { // Returns min l in [0, r] where
    g(query(l, r)) is true.
    if (r < 0 || r > n - 1) { return r + 1; }
    S acc = Mon::idS();
    lng res = _minL(roots[ver], 0, n - 1, r, g, acc, Mon::idF());
    return res == -1 ? 0 : res;
}

S _query(int i, lng l, lng r, lng ql, lng qr, F acc) {
    if (i == -1) {
        if (max(l, ql) > min(r, qr)) { return Mon::idS(); }
        return Mon::map(acc, Mon::defR(max(l, ql), min(r, qr)));
    }
    if (ql <= l && r <= qr) { return Mon::map(acc, tree[i].val); }
    F nac = Mon::cmp(acc, tree[i].lzy);
    lng md = std::midpoint(l, r);
    if (qr <= md) { return _query(tree[i].lc, l, md, ql, qr,
        nac); }
    if (ql > md) { return _query(tree[i].rc, md + 1, r, ql, qr,
        nac); }
    return Mon::ope(_query(tree[i].lc, l, md, ql, qr, nac),
        _query(tree[i].rc, md + 1, r, ql, qr, nac));
}

S query(int ver, lng l, lng r) {
    l = max(l, 0LL); r = min(r, n - 1);
    if (l > r) { return Mon::idS(); }
    return _query(roots[ver], 0, n - 1, l, r, Mon::idF());
}

```

2.8 Kth Smallest Persistent Segment Tree

```

    if (l_lc == -1) { cnt -= segt->tree[l_lc].val.cnt; }
    int r_lc = r_i == -1 ? -1 : segt->tree[r_i].lc;
    if (r_lc != -1) { cnt += segt->tree[r_lc].val.cnt; }
    int md = std::midpoint(l, r);
    if (cnt >= k) { return _kthMin(l_lc, r_lc, l, md, k); }
    int l_rc = l_i == -1 ? -1 : segt->tree[l_i].rc;
    int r_rc = r_i == -1 ? -1 : segt->tree[r_i].rc;
    return _kthMin(l_lc, r_rc, md + 1, r, k - cnt);
}

lng kthMin(int l, int r, int k) {
    assert(0 <= l && l <= r && r < n);
    assert(1 <= k && k <= r - l + 1);
    int ord = _kthMin(segt->roots[l], segt->roots[r + 1], 0,
        v.size() - 1, k);
    return v[ord];
}

lng _kthSum(int l_i, int r_i, int l, int r, int k) {
    if (k == 0) { return 0; }
    if (l == r) { return lng(k) * v[l]; }
    int l_cnt = 0; lng l_sum = 0;
    int l_lc = l_i == -1 ? -1 : segt->tree[l_i].lc;
    if (l_lc != -1) {
        l_cnt -= segt->tree[l_lc].val.cnt;
        l_sum -= segt->tree[l_lc].val.sum;
    }
    int r_lc = r_i == -1 ? -1 : segt->tree[r_i].lc;
    if (r_lc != -1) {
        l_cnt += segt->tree[r_lc].val.cnt;
        l_sum += segt->tree[r_lc].val.sum;
    }
    int md = std::midpoint(l, r);
    if (l_cnt >= k) { return _kthSum(l_lc, r_lc, l, md, k); }
    else {
        int l_rc = l_i == -1 ? -1 : segt->tree[l_i].rc;
        int r_rc = r_i == -1 ? -1 : segt->tree[r_i].rc;
        return l_sum + _kthSum(l_rc, r_rc, md + 1, r, k - l_cnt);
    }
}

lng kthSum(int l, int r, int k) {
    assert(0 <= l && l <= r && r < n);
    assert(0 <= k && k <= r - l + 1);
    return _kthSum(segt->roots[l], segt->roots[r + 1], 0, v.size()
        - 1, k);
}

lng maxLeq(int l, int r, lng x) {
    assert(0 <= l && l <= r && r < n);
    int cnt = _cntLeq(l, r, x);
    if (cnt == 0) { return -INF64; }
    return kthMin(l, r, cnt);
}

lng minGeq(int l, int r, lng x) {
    assert(0 <= l && l <= r && r < n);
    int cnt = _cntLeq(l, r, x - 1);
    if (cnt == r - l + 1) { return INF64; }
    return kthMin(l, r, cnt + 1);
}
}

};

2.9 Segment Tree Beats

// S: O(n), U: O(log(n)^2), Q: O(log(n)), M: O(n)
struct SegTreeBeats {
    struct Node {
        lng sum;
        lng mx1, mx2; int mxc;
        lng mn1, mn2; int mnc;
        bool to_set; lng add_upd, set_upd;
    };
    int n;
    vector<Node> tree;

    template<typename T>

```

2.9 Segment Tree Beats

```
// S: O(n), U: O(log(n)^2), Q: O(log(n)), M: O(n)
struct SegTreeBeats {
    struct Node {
        lng sum;
        lng mx1, mx2; int mxc;
        lng mn1, mn2; int mnc;
        bool to_set; lng add_upd, set_upd;
    };
    int n;
    vector<Node> tree;

    template<typename T>
```

```

SegTreeBeats<std::span<const T> &a>: n(a.size()),
tree(2 * std::bit_ceil(static_cast<unsigned>(max(n, 1)))) {
    if (n > 0) { build(1, 0, n - 1, a); }
}

template<typename T>
void build(int i, int l, int r, std::span<const T> &a) {
    if (l == r) {
        tree[i].sum = a[l];
        tree[i].mx1 = a[l]; tree[i].mx2 = -INF64; tree[i].mxc = 1;
        tree[i].mn1 = a[l]; tree[i].mn2 = INF64; tree[i].mnc = 1;
        tree[i].to_set = false; tree[i].add_upd = 0;
        tree[i].set_upd = 0;
        return;
    }
    int md = std::midpoint(l, r);
    build(2 * i, l, md, a);
    build(2 * i + 1, md + 1, r, a);
    pull(i);
    tree[i].to_set = false; tree[i].add_upd = 0; tree[i].set_upd = 0;
}

inline void pull(int i) {
    int lc = 2 * i, rc = 2 * i + 1;
    tree[i].sum = tree[lc].sum + tree[rc].sum;
    if (tree[lc].mx1 == tree[rc].mx1) {
        tree[i].mx1 = tree[lc].mx1;
        tree[i].mx2 = max(tree[lc].mx2, tree[rc].mx2);
        tree[i].mxc = tree[lc].mxc + tree[rc].mxc;
    } else if (tree[lc].mx1 > tree[rc].mx1) {
        tree[i].mx1 = tree[lc].mx1;
        tree[i].mx2 = max(tree[lc].mx2, tree[rc].mx1);
        tree[i].mxc = tree[lc].mxc;
    } else {
        tree[i].mx1 = tree[rc].mx1;
        tree[i].mx2 = max(tree[lc].mx1, tree[rc].mx2);
        tree[i].mxc = tree[rc].mxc;
    }
    if (tree[lc].mn1 == tree[rc].mn1) {
        tree[i].mn1 = tree[lc].mn1;
        tree[i].mn2 = min(tree[lc].mn2, tree[rc].mn2);
        tree[i].mnc = tree[lc].mnc + tree[rc].mnc;
    } else if (tree[lc].mn1 < tree[rc].mn1) {
        tree[i].mn1 = tree[lc].mn1;
        tree[i].mn2 = min(tree[lc].mn2, tree[rc].mn1);
        tree[i].mnc = tree[lc].mnc;
    } else {
        tree[i].mn1 = tree[rc].mn1;
        tree[i].mn2 = min(tree[lc].mn1, tree[rc].mn2);
        tree[i].mnc = tree[rc].mnc;
    }
}

inline void addApply(int i, int len, lng x) {
    tree[i].sum += x * len;
    tree[i].mx1 += x; if (tree[i].mx2 != -INF64) { tree[i].mx2 += x; }
    tree[i].mn1 += x; if (tree[i].mn2 != INF64) { tree[i].mn2 += x; }
    if (tree[i].to_set) { tree[i].set_upd += x; }
    else { tree[i].add_upd += x; }
}

inline void chmaxApply(int i, lng x) {
    if (x <= tree[i].mn1) { return; }
    tree[i].sum += (x - tree[i].mn1) * tree[i].mnc;
    if (tree[i].mx1 == tree[i].mn1) { tree[i].mx1 = x; }
    if (tree[i].mx2 == tree[i].mn1) { tree[i].mx2 = x; }
    tree[i].mn1 = x;
}

inline void chminApply(int i, lng x) {
    if (x >= tree[i].mx1) { return; }
    tree[i].sum -= (tree[i].mx1 - x) * tree[i].mxc;
    if (tree[i].mn1 == tree[i].mx1) { tree[i].mn1 = x; }
}

```

```

void addUpdate(int l, int r, lng x) {
    l = max(l, 0); r = min(r, n - 1);
    if (l <= r) { _addUpdate(1, 0, n - 1, l, r, x); }
}

void chminUpdate(int l, int r, lng x) {
    l = max(l, 0); r = min(r, n - 1);
    if (l <= r) { _chminUpdate(1, 0, n - 1, l, r, x); }
}

void chmaxUpdate(int l, int r, lng x) {
    l = max(l, 0); r = min(r, n - 1);
    if (l <= r) { _chmaxUpdate(1, 0, n - 1, l, r, x); }
}

void setUpdate(int l, int r, lng x) {
    l = max(l, 0); r = min(r, n - 1);
    if (l <= r) { _setUpdate(1, 0, n - 1, l, r, x); }
}

lng _maxQuery(int i, int l, int r, int ql, int qr) {
    if (ql <= l && r <= qr) { return tree[i].mx1; }
    push(i, l, r);
    int md = std::midpoint(l, r);
    lng res = -INF64;
    if (ql <= md) { res = max(res, _maxQuery(2 * i, l, md, ql, qr)); }
    if (qr > md) { res = max(res, _maxQuery(2 * i + 1, md + 1, r, ql, qr)); }
    return res;
}

lng _minQuery(int i, int l, int r, int ql, int qr) {
    if (ql <= l && r <= qr) { return tree[i].mn1; }
    push(i, l, r);
    int md = std::midpoint(l, r);
    lng res = INF64;
    if (ql <= md) { res = min(res, _minQuery(2 * i, l, md, ql, qr)); }
    if (qr > md) { res = min(res, _minQuery(2 * i + 1, md + 1, r, ql, qr)); }
    return res;
}

lng _sumQuery(int i, int l, int r, int ql, int qr) {
    if (ql <= l && r <= qr) { return tree[i].sum; }
    push(i, l, r);
    int md = std::midpoint(l, r);
    lng res = 0;
    if (ql <= md) { res += _sumQuery(2 * i, l, md, ql, qr); }
    if (qr > md) { res += _sumQuery(2 * i + 1, md + 1, r, ql, qr); }
    return res;
}

lng maxQuery(int l, int r) {
    l = max(l, 0); r = min(r, n - 1);
    if (l > r) { return -INF64; }
    return _maxQuery(1, 0, n - 1, l, r);
}

lng minQuery(int l, int r) {
    l = max(l, 0); r = min(r, n - 1);
    if (l > r) { return INF64; }
    return _minQuery(1, 0, n - 1, l, r);
}

lng sumQuery(int l, int r) {
    l = max(l, 0); r = min(r, n - 1);
    if (l > r) { return 0; }
    return _sumQuery(1, 0, n - 1, l, r);
}
}

```

3.1 Point

```
// T: O(1), M: O(1)
vector<point> getIntsLineCircle(double r, double a, double b, double
c) { // ax + by + c = 0
    double d_sq = c * c / (a * a + b * b), x0 = -a * c / (a * a + b *
b), y0 = -b * c / (a * a + b * b);
    if (d_sq > r * r + 1e-9) { return {}; }
    if (std::abs(d_sq - r * r) < 1e-9) { return {dpoint(x0, y0)}; }
    double d = std::sqrt((r * r - d_sq) / (a * a + b * b));
    return {dpoint(x0 + b * d, y0 - a * d), dpoint(x0 - b * d, y0 + a *
d)};
}
```

```

return ps[i].x < ps[j].x || (ps[i].y == ps[j].x && ps[i].y <
ps[j].y);
});

auto mul = [&](lng a, lng b) -> lng { return (a - b) * (a - b); };
auto update = [&](int i, int j) -> void {
    lng d = mul(ps[i].x, ps[j].x) + mul(ps[i].y, ps[j].y);
    if (d < mn_d) { mn_d = d; res = {i, j}; }
};

auto solve = [&](auto &&solve, int l, int r) -> void {
    if (r - l <= 3) {
        for (int i = l; i < r; i++) {
            for (int j = i + 1; j < r; j++) {
                update(ids[i], ids[j]);
            }
            std::sort(ids.begin() + l, ids.begin() + r,
                [&](int i, int j) { return ps[i].y < ps[j].y; });
        }
    }

    int md = std::midpoint(l, r); lng md_x = ps[ids[md]].x;
    solve(solve, l, md);
    solve(solve, md, r);
    std::merge(ids.begin() + l, ids.begin() + md, ids.begin() + md,
        ids.begin() + r, tmp.begin(),
        [&](int i, int j) { return ps[i].y < ps[j].y; });
    std::copy(tmp.begin(), tmp.begin() + r - l, ids.begin() + l);

    int ti = 0;
    for (int i = l; i < r; i++) {
        if (mul(ps[ids[i]].x, md_x) < mn_d) {
            for (int j = ti - 1; j >= 0 && mul(ps[ids[i]].y,
                ps[tmp[j]].y) < mn_d; j--) {
                update(ids[i], tmp[j]);
            }
            tmp[ti++] = ids[i];
        }
    }
};

solve(solve, 0, n);
return res;
}

```

3.6 Get Minimum Enclosing Circle

```
// S: O(n), U: NA, Q: O(1), M: O(n)
struct WeizlMEC {
    using cdb = std::complex<double>;
    using mec = std::variant<array<cdb, 2>, array<cdb, 3>>;
    static inline std::mt19937_64
    rng{std::chrono::steady_clock::now().time_since_epoch().count()};

    double indicator(const mec &c, cdb p) {
        return std::visit([&](auto &&c) -> double {
            cdb a = c[0], b = c[1], i0 = (b - p) * std::conj(a - p);
            if constexpr (std::tuple_size_v<std::decay_t<decltype(c)>>
                == 2) {
                return std::real(i0);
            } else {
                cdb i2 = (a - c[2]) * std::conj(b - c[2]), i1 = i0 *
                i2;
                return std::imag(i2) < 0 ? -std::imag(i1) :
                std::imag(i1);
            }
        }, c);
    }
}
```

```
bool isInside(const mec &c, cdb p) { return indicator(c, p) <= 0; }
```

```

mec getMEC(vector<cdb> ps) {
    if (ps.empty()) { return array<cdb, 2>{0, 0}; }
    if (ps.size() == 1) { return array<cdb, 2>{ps[0], ps[0]}; }

    std::shuffle(ps.begin(), ps.end(), rng);
    mec c = array<cdb, 2>{ps[0], ps[1]};
    for (int i = 0; i < ps.size(); i++) {
        if (!isInside(c, ps[i])) { c = array<cdb, 2>{ps[i], ps[0]}; }
        for (int j = 0; j < i; j++) {
            if (!isInside(c, ps[j])) { c = array<cdb, 2>{ps[i], ps[j]}; }
            for (int k = 0; k < j; k++) {
                if (!isInside(c, ps[k])) { c = array<cdb, 3>{ps[i], ps[j],
                    ps[k]}; }
            }
        }
    }
    return c;
}
};

```

3.8 Li-Chao Tree

```
// S: O(n), U: O(log(n)) or O(log(n)^2), Q: O(log(n)), M: O(n * log(n))
struct LiChaoTree {
    struct Line {
        lng m, b;
        lng eval(lng x) const { return m * x + b; }
    };
    struct Node {
        Line mx_l, mn_l; int lc = -1, rc = -1;
        Node(Line Mx_l, Line Mn_l) : mx_l(Mx_l), mn_l(Mn_l) {}
    };
    lng mn_x, mx_x;
    int root = -1;
    vector<Node> tree;

    LiChaoTree(lng Mn_x, lng Mx_x) : mn_x(Mn_x), mx_x(Mx_x) {
        tree.reserve(2'500'000); }
    int createNode() {
        tree.emplace_back(Line{0, -INF64}, Line{0, INF64});
        return tree.size() - 1;
    }

    int _addLineMax(int i, lng l, lng r, Line y) {
        if (i == -1) { i = createNode(); }
        lng md = std::midpoint(l, r);
        bool l_bet = y.eval(l) > tree[i].mx_l.eval(l);
        bool m_bet = y.eval(md) > tree[i].mx_l.eval(md);
        if (m_bet) { std::swap(tree[i].mx_l, y); }
        if (l == r) { return i; }
    }
};
```

```

    if (l_bet != m_bet) { int nlc = _addLineMax(tree[i].lc, l,
md, y); tree[i].lc = nlc; }
    else { int nrc = _addLineMax(tree[i].rc, md + 1,
r, y); tree[i].rc = nrc; }
    return i;
}
int _addLineMin(int i, lng l, lng r, Line y) {
    if (i == -1) { i = createNode(); }
    lng md = std::midpoint(l, r);
    bool l_bet = y.eval(l) < tree[i].mn_l.eval(l);
    bool m_bet = y.eval(md) < tree[i].mn_l.eval(md);
    if (m_bet) { std::swap(tree[i].mn_l, y); }
    if (l == r) { return i; }
    if (l_bet != m_bet) { int nlc = _addLineMin(tree[i].lc, l,
md, y); tree[i].lc = nlc; }
    else { int nrc = _addLineMin(tree[i].rc, md + 1,
r, y); tree[i].rc = nrc; }
    return i;
}
int _addSegmMax(int i, lng l, lng r, lng ql, lng qr, Line y) {
    if (ql <= l && r <= qr) { return _addLineMax(i, l, r, y); }
    if (i == -1) { i = createNode(); }
    lng md = std::midpoint(l, r);
    if (ql <= md) { int nlc = _addSegmMax(tree[i].lc, l, md,
ql, qr, y); tree[i].lc = nlc; }
    if (qr > md) { int nrc = _addSegmMax(tree[i].rc, md + 1, r,
ql, qr, y); tree[i].rc = nrc; }
    return i;
}
int _addSegmMin(int i, lng l, lng r, lng ql, lng qr, Line y) {
    if (ql <= l && r <= qr) { return _addLineMin(i, l, r, y); }
    if (i == -1) { i = createNode(); }
    lng md = std::midpoint(l, r);
    if (ql <= md) { int nlc = _addSegmMin(tree[i].lc, l, md,
ql, qr, y); tree[i].lc = nlc; }
    if (qr > md) { int nrc = _addSegmMin(tree[i].rc, md + 1, r,
ql, qr, y); tree[i].rc = nrc; }
    return i;
}
void addLine(lng m, lng b) {
    root = _addLineMax(root, mn_x, mx_x, {m, b});
    root = _addLineMin(root, mn_x, mx_x, {m, b});
}
void addSegm(lng l, lng r, lng m, lng b) {
    if (l >= mn_x && l <= r && r <= mx_x) {
        root = _addSegmMax(root, mn_x, mx_x, l, r, {m, b});
        root = _addSegmMin(root, mn_x, mx_x, l, r, {m, b});
    }
}
lng _maxQuery(int i, lng l, lng r, lng x) {
    if (i == -1) { return -INF64; }
    lng res = tree[i].mx_l.eval(x);
    if (l == r) { return res; }
    lng md = std::midpoint(l, r);
    if (x <= md) { res = max(res, _maxQuery(tree[i].lc, l, md,
x)); }
    else { res = max(res, _maxQuery(tree[i].rc, md + 1, r,
x)); }
    return res;
}
lng _minQuery(int i, lng l, lng r, lng x) {
    if (i == -1) { return INF64; }
    lng res = tree[i].mn_l.eval(x);
    if (l == r) { return res; }
    lng md = std::midpoint(l, r);
    if (x <= md) { res = min(res, _minQuery(tree[i].lc, l, md,
x)); }
    else { res = min(res, _minQuery(tree[i].rc, md + 1, r,
x)); }
    return res;
}
}

```

```

lng maxQuery(lng x) {
    if (x < mn_x || x > mx_x) { return -INF64; }
    return _maxQuery(root, mn_x, mx_x, x);
}
lng minQuery(lng x) {
    if (x < mn_x || x > mx_x) { return INF64; }
    return _minQuery(root, mn_x, mx_x, x);
}
}
}
4 Graphs
4.1 Topological Sort
void toposort(int u) {
    visited[u] = true;
    for (auto v : graph[u])
        if (!visited[v]) {
            toposort(v);
        }
    stack.push(u); // the key difference between dfs and topological
sort
}
}

```

4.2 Cycle Find DFS

```

bool dfs(int u) {
    color[u] = 1;
    for (int v : adj[u]) {
        if (color[v] == 0) {
            if (dfs(v))
                return true;
        } else if (color[v] == 1) {
            cycles.push_back({v, u});
            return true;
        }
    }
    color[u] = 2;
    return false;
}
void find_cycles() {
    for (int u = 0; u < n; u++) {
        if (color[u] == 0)
            dfs(u);
    }
    for (int i = 0; i < cycles.size(); i++) {
        auto& [start, end] = cycles[i];
        for (int v = end; v != start; v = prev[v]) {
            cycle_grp[v] = i+1;
        }
        auto it = find(adj[end].begin(), adj[end].end(), start);
        adj[end].erase(it);
        cycle_grp[start] = i+1;
    }
}
// S: O(n), U: O(a(n)), Q: O(a(n)), M: O(n)
struct DSU {
    int n, ncon;
    vector<int> par, siz, esz;
    vector<lng> dis;
    vector<bool> is_bip;
    DSU(int N) : n(N), ncon(N), par(n, 0), siz(n, 1), esz(n, 0), dis(n,
0), is_bip(n, true) {
        iota(par.begin(), par.end(), 0);
    }
}

```

4.3 Disjoint Set Union/UnionFind

```

bool uniteSets(int u, int v, lng w = 1) {
    int ru = findSet(u); int rv = findSet(v);
    if (ru == rv) {
        esz[ru]++;
        if ((dis[u] - dis[v] - w) & 1) { is_bip[ru] = false; }
        return false;
    }
    if (siz[ru] < siz[rv]) { std::swap(ru, rv); std::swap(u, v); w
= -w; }
    ncon--;
    par[rv] = ru; siz[ru] += siz[rv]; esz[ru] += esz[rv] + 1;
    dis[rv] = dis[u] - dis[v] - w;
    is_bip[ru] = is_bip[ru] && is_bip[rv];
    return true;
}
int findSet(int u) {
    if (u == par[u]) { return u; }
    int p = par[u];
    par[u] = findSet(p);
    dis[u] += dis[p];
    return par[u];
}
bool isSameSet(int u, int v) { return findSet(u) == findSet(v); }
}
}
4.4 Shortest Paths
4.4.1 Dijkstra
struct Dijkstra {
    int n;
    vector<bool> is_proc;
    vector<lng> dis;
    priority_queue<pair<lng, int>, vector<pair<lng, int>>,
greater<pair<lng, int>>> unproc;
    Dijkstra(int n) : n(n) {
        is_proc.resize(n + 1);
        dis.resize(n + 1, INF64);
    }
    void runDijkstra(const vector<int> &src, const vector<vector<int>>
&adjl) {
        fill(is_proc.begin(), is_proc.end(), false);
        fill(dis.begin(), dis.end(), INF64);
        priority_queue<pair<lng, int>, vector<pair<lng, int>>,
greater<pair<lng, int>>> empty;
        unproc.swap(empty);
        for (int u : src) {
            dis[u] = 0;
            unproc.push({dis[u], u});
        }
        while (!unproc.empty()) {
            int cur = unproc.top().se;
            unproc.pop();
            if (is_proc[cur]) {
                continue;
            }
            is_proc[cur] = true;
            for (int nxt : adjl[cur]) {
                if (dis[cur] + 1 < dis[nxt]) {
                    dis[nxt] = dis[cur] + 1;
                    unproc.push({dis[nxt], nxt});
                }
            }
        }
    }
};
}

```

```

void runDijkstra(const vector<int> &src, const
vector<vector<pair<int, lng>>> &adjl) {
    fill(is_proc.begin(), is_proc.end(), false);
    fill(dis.begin(), dis.end(), INF64);
    priority_queue<pair<lng, int>, vector<pair<lng, int>>,
greater<pair<lng, int>>> empty;
    unproc.swap(empty);

    for (int u : src) {
        dis[u] = 0;
        unproc.push({dis[u], u});
    }
    while (!unproc.empty()) {
        int cur = unproc.top().se;
        unproc.pop();

        if (is_proc[cur]) {
            continue;
        }
        is_proc[cur] = true;

        for (auto [nxt, w] : adjl[cur]) {
            if (dis[cur] + w < dis[nxt]) {
                dis[nxt] = dis[cur] + w;
                unproc.push({dis[nxt], nxt});
            }
        }
    }
};

```

4.4.2 Bellman-Ford

```

void bellman_ford(int src) {
    dist[src] = 0;

    for (int k = 0; k < n-1; k++) {
        for (int u = 0; u < n; u++) {
            for (auto e : adj[u]) {
                int v = e.second;
                ll w = e.first;
                if (dist[u] < INF) { // if graph has negative weights
                    dist[v] = min(dist[v], dist[u] + w);
                }
            }
        }
    }

    bool has_negative_cycle() { // call after running bellman-ford
        for (int u = 0; u < n; u++) {
            for (auto e : adj[u]) {
                int v = e.second;
                ll w = e.first;
                if (dist[v] > dist[u] + w)
                    return true;
            }
        }
        return false;
    }
};

```

4.4.3 Floyd-Warshall

```

void floyd_warshall() {
    for (int k = 0; k < n; k++) {
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j]);
            }
        }
    }
};

```

4.5 Trees

4.5.1 Lowest Common Ancestor

```

// S: O(n * log(n)), U: NA, Q: O(log(n)), M: O(n * log(n))
struct LCA {
    int n, l, root, tm;
    vector<int> dep, tin, tout;
    vector<vector<int>> up;

    LCA(int n, int rt, const vector<vector<int>> &adj) :
        n(n), root(rt), tm(0), dep(n, 0), tin(n, 0), tout(n, 0) {
        l = std::ceil(std::log2(max(n, 1)));
        up.assign(n, vector<int>(l + 1));

        auto dfs = [&](auto &&dfs, int u, int p, int d) -> void {
            dep[u] = d; tin[u] = ++tm;
            up[u][0] = p;
            for (int i = 1; i <= l; i++) { up[u][i] = up[up[u][i - 1]][i - 1]; }
            for (int v : adj[u]) {
                if (v != p) { dfs(dfs, v, u, d + 1); }
            }
            tout[u] = ++tm;

            dfs(dfs, root, root, 0);
        };

        bool isAnc(int u, int v) const { return tin[u] <= tin[v] && tout[u]
>= tout[v]; }
        int getDis(int u, int v) const { return dep[u] + dep[v] - 2 *
dep[getLCA(u, v)]; }
        int getKthAncestor(int u, int k) const {
            if (k <= 0) { return u; }
            if (k > dep[u]) { return -1; }
            for (int i = l; i >= 0; i--) {
                if (k & (1 << i)) { u = up[u][i]; }
            }
            return u;
        }
        int getLCA(int u, int v) const {
            if (isAnc(u, v)) { return u; }
            if (isAnc(v, u)) { return v; }
            for (int i = l; i >= 0; i--) {
                if (!isAnc(up[u][i], v)) { u = up[u][i]; }
            }
            return up[u][0];
        }
};

```

4.5.2 Heavy Light Decomposition

```

// S: O(n), U: O(log(n)) or O(log(n)^2), Q: O(log(n)) or O(log(n)^2), M:
O(n)
struct HLD {
    int n, tm = 0;
    vector<int> dep, par, siz, head, in, out, rev;
    vector<vector<int>> adj;

    HLD(int N) : n(N), dep(N), par(N, -1), siz(N), head(N), in(N),
out(N), rev(N), adj(N) {}

    void addEdge(int u, int v) { adj[u].push_back(v);
adj[v].push_back(u); }
    void build(int root = 0) { tm = 0; dfsSiz(root, -1, 0);
dfsHld(root, -1, root); }
    void dfsSiz(int u, int p, int d) {
        dep[u] = d; par[u] = p; siz[u] = 1;
        if (adj[u].size() > 1 && adj[u][0] == p) { std::swap(adj[u][0],
adj[u][1]); }
        for (int &v : adj[u]) {
            if (v == p) { continue; }
            dfsSiz(v, u, d + 1);
            siz[u] += siz[v];
            if (siz[v] > siz[adj[u][0]]) { std::swap(v, adj[u][0]); }
        }
    }
};

```

```

}
void dfsHld(int u, int p, int h) {
    head[u] = h; in[u] = tm++; rev[in[u]] = u;
    for (int v : adj[u]) {
        if (v == p) { continue; }
        dfsHld(v, u, (v == adj[u][0] ? h : v));
    }
    out[u] = tm - 1;
}

int getDis(int u, int v) { return dep[u] + dep[v] - 2 *
dep[getLCA(u, v)]; }
int getLCA(int u, int v) {
    while (head[u] != head[v]) { std::swap(u, v); }
    if (dep[head[u]] < dep[head[v]]) { std::swap(u, v); }
    u = par[head[u]];
    return dep[u] < dep[v] ? u : v;
}
int kthAnc(int u, int k) {
    if (dep[u] < k) { return -1; }
    while (dep[u] - dep[head[u]] < k) {
        k -= dep[u] - dep[head[u]] + 1;
        u = par[head[u]];
    }
    return rev[in[u] - k];
}

template<typename F>
void processPath(int u, int v, bool is_edge, F f) {
    while (head[u] != head[v]) {
        if (dep[head[u]] < dep[head[v]]) { std::swap(u, v); }
        f(in[head[u]], in[u]);
        u = par[head[u]];
    }
    if (dep[u] > dep[v]) { std::swap(u, v); }
    if (u == v && is_edge) { return; }
    f(in[u] + is_edge, in[v]);
}

template<typename F>
void processSubt(int u, bool is_edge, F f) {
    if (in[u] == out[u] && is_edge) { return; }
    f(in[u] + is_edge, out[u]);
}

template<typename T>
vector<T> reorderArray(const vector<T> &a) {
    vector<T> res(n);
    for (int i = 0; i < n; i++) { res[in[i]] = a[i]; }
    return res;
}
};

```

4.5.3 Centroid Decomposition

```

struct tree {
    int n;
    vector<vector<int>> adj;
    vector<int> sz;
    vector<bool> done;

    tree(int n) : n(n) {
        adj.resize(n, {});
        sz.resize(n);
        done.resize(n);
    }

    int get_size(int u, int par = 0) {
        sz[u] = 1;
        for (int v : adj[u]) {
            if (!done[v] and v != par)
                sz[u] += get_size(v, u);
        }
        return sz[u];
    }
};

```

```

}
int get_centroid(int target, int u, int par = 0) {
    for (int v : adj[u])
        if (!done[v] and v != par and sz[v] >= target)
            return get_centroid(target, v, u);
    return u;
}
void centroid_decomp(int root = 0) {
    int u = get_centroid(get_size(root)/2, root);

    // do something

    done[u] = true;

    for (int v : adj[u]) {
        if (!done[v])
            centroid_decomp(v);
    }
}
};

```

4.6 Connectivity and Components

4.6.1 Bridge-Finding Algorithm

```

struct BridgeAlgo {
    int n, timer = 0;
    vector<bool> vst;
    vector<int> t_in, low;
    set<pair<int, int>> bridges;

    BridgeAlgo(int n, const vector<vector<int>> &adjl): n(n) {
        vst.resize(n);
        t_in.resize(n, -1);
        low.resize(n, -1);

        for (int i = 0; i < n; i++) {
            if (!vst[i]) {
                dfs(i, i, adjl);
            }
        }

        void dfs(int cur, int prv, const vector<vector<int>> &adjl) {
            vst[cur] = true;
            t_in[cur] = timer;
            low[cur] = timer;
            timer++;

            bool prv_skipped = false;
            for (int nxt : adjl[cur]) {
                if (nxt == prv && !prv_skipped) {
                    prv_skipped = true;
                    continue;
                }

                if (vst[nxt]) {
                    low[cur] = min(low[cur], t_in[nxt]);
                } else {
                    dfs(nxt, cur, adjl);

                    low[cur] = min(low[cur], low[nxt]);
                    if (low[nxt] > t_in[cur]) {
                        bridges.insert({min(cur, nxt), max(cur, nxt)});
                    }
                }
            }
        }
};

```

4.6.2 Block-Cut Tree

```

struct tree {
    int n;

```

```

    vector<int> is_cut, id;
    vector<vector<int>> adj;

    tree(int _n, vector<int> &cutpoints, vector<vector<int>> &comps) {
        n = cutpoints.size() + comps.size();
        l = ceil(log2(n));
        adj.resize(n);
        id.resize(_n);
        is_cut.resize(_n, false);

        for (int i = 0; i < cutpoints.size(); i++) {
            id[cutpoints[i]] = i;
            is_cut[cutpoints[i]] = true;
        }
        for (int i = 0; i < comps.size(); i++) {
            int _id = i + cutpoints.size();
            int (int u : comps[i]) {
                if (is_cut[u])
                    add_edge(id[u], _id);
                else
                    id[u] = _id;
            }
        }
        void add_edge(int u, int v) {
            adj[u].push_back(v);
            adj[v].push_back(u);
        }
    }

    4.6.3 Cutpoints-Bridges

    struct graph {
        int n, timer;
        vector<vector<int>> adj, comps;
        vector<int> tin, low, vis, stk, cutpoints, bridges;

        graph(int n) : n(n) {
            adj.resize(n);

            void add_edge(int u, int v) {
                adj[u].push_back(v);
                adj[v].push_back(u);
            }

            void dfs(int u, int p = -1) {
                vis[u] = true;
                tin[u] = low[u] = timer++;
                stk.push_back(u);
                int children = 0;
                bool has_low_child = false;

                for (int v : adj[u]) {
                    if (v == p)
                        continue;
                    if (vis[v])
                        low[u] = min(low[u], tin[v]);
                    else {
                        dfs(v, u);
                        children++;
                        low[u] = min(low[u], low[v]);
                        if (low[v] > tin[u])
                            bridges.push_back(u);
                        if (low[v] >= tin[u]) {
                            has_low_child = true;
                            comps.push_back({u});
                            while (comps.back().back() != v and !stk.empty()) {
                                comps.back().push_back(stk.back());
                                stk.pop_back();
                            }
                        }
                    }
                }
            }
        }
    }
}

```

```

        if ((p == -1 and children >= 2) or (p != -1 and has_low_child))
            cutpoints.push_back(u);
    }

    void find_cutpoints_bridges() {
        vis.assign(n, false);
        tin.assign(n, -1);
        low.assign(n, -1);
        cutpoints.clear();
        bridges.clear();
        comps.clear();
        stk.clear();
        timer = 0;
        for (int u = 0; u < n; u++)
            if (!vis[u])
                dfs(u);
    }
}

```

4.6.4 Kosaraju

```

struct graph {
    int n;
    vector<bool> vis;
    vector<vector<vector<int>>> adj;
    vector<vector<int>> comps;

    graph(int n) : n(n) {
        vis.resize(n);
        adj.resize(2);
        for (int dir = 0; dir < 2; dir++)
            adj[dir] = vector<vector<int>> (n);
    }

    void add_edge(int u, int v) {
        adj[0][u].push_back(v);
        adj[1][v].push_back(u);
    }

    void dfs(int u, int p, int dir, vector<int> &topo) {
        vis[u] = true;
        for (int v : adj[dir][u])
            if (!vis[v] and v != p)
                dfs(v, u, dir, topo);
        topo.push_back(u);
    }

    void kosaraju() {
        vector<int> topo;
        vis.assign(n, false);
        for (int u = 0; u < n; u++)
            if (!vis[u])
                dfs(u, -1, 0, topo);
        vis.assign(n, false);
        for (int i = n-1; i >= 0; i--) {
            if (!vis[topo[i]]) {
                comps.push_back({});
                dfs(topo[i], -1, 1, comps.back());
            }
        }
    }
}

```

4.6.5 2SAT

```

#include <bits/stdc++.h>

using namespace std;

struct two_sat {
    int n;
    vector<vector<int>> g, gr;
    vector<int> comp, topological_order, answer;

```

```

vector<bool> vis;
two_sat() {}
two_sat(int _n) { init(_n); }
void init(int _n) {
    n = _n;
    g.assign(2 * n, vector<int>());
    gr.assign(2 * n, vector<int>());
    comp.resize(2 * n);
    vis.resize(2 * n);
    answer.resize(2 * n);
}

void add_edge(int u, int v) {
    g[u].push_back(v);
    gr[v].push_back(u);
}

// At least one of them is true
void add_clause_or(int i, bool f, int j, bool g) {
    add_edge(i + (f ? n : 0), j + (g ? 0 : n));
    add_edge(j + (g ? n : 0), i + (f ? 0 : n));
}

// Only one of them is true
void add_clause_xor(int i, bool f, int j, bool g) {
    add_clause_or(i, f, j, g);
    add_clause_or(i, !f, j, !g);
}

// Both of them have the same value
void add_clause_and(int i, bool f, int j, bool g) {
    add_clause_xor(i, !f, j, g);
}

void dfs(int u) {
    vis[u] = true;
    for (const auto &v : g[u])
        if (!vis[v]) dfs(v);
    topological_order.push_back(u);
}

void scc(int u, int id) {
    vis[u] = true;
    comp[u] = id;
    for (const auto &v : gr[u])
        if (!vis[v]) scc(v, id);
}

bool satisfiable() {
    fill(vis.begin(), vis.end(), false);
    for (int i = 0; i < 2 * n; i++)
        if (!vis[i]) dfs(i);
    fill(vis.begin(), vis.end(), false);
    reverse(topological_order.begin(), topological_order.end());
    int id = 0;
    for (const auto &v : topological_order)
        if (!vis[v]) scc(v, id++);
    for (int i = 0; i < n; i++) {
        if (comp[i] == comp[i + n]) return false;
        answer[i] = (comp[i] > comp[i + n] ? 1 : 0);
    }
    return true;
}

int main() {
    int N, M;
    cin >> N >> M;
    two_sat solver(M); // 0 index
    for (int i = 0; i < N; i++){
        int t1, t2;
        char p1, p2;
        cin >> p1 >> t1 >> p2 >> t2; //2nd/4th are bool if cond. is
        true

```

```

        solver.add_clause_or(t1 - 1, p1 == '+', t2 - 1, p2 == '+');
    }
    if (!solver.satisfiable()) {
        cout << "IMPOSSIBLE";
        return 0;
    }
    for (int i = 0; i < M; i++)
        cout << (solver.answer[i] ? '+' : '-') << " ";
}

```

4.6.6 Flows and Bipartite Matching

4.6.7 Dinic

```

struct edge {
    int u, v;
    ll c, f;
    ll resid() { return c - f; }
};

struct flow_network {
    vector<edge> edges;
    vector<vector<int>> adj;
    vector<int> level, ptr;
    queue<int> q;
    int n, s, t;
    flow_network(int n, int s, int t) : n(n), s(s), t(t) {
        adj.resize(n);
        level.resize(n);
        ptr.resize(n);
    }
    void add_edge(int u, int v, ll cap) {
        edges.push_back({u, v, cap, 0});
        adj[u].push_back(len(edges)-1);
        edges.push_back({v, u, 0, 0});
        adj[v].push_back(len(edges)-1);
    }
    bool bfs() {
        while (!q.empty()) {
            int u = q.front(); q.pop();
            for (int id : adj[u]) {
                if (edges[id].c == edges[id].f)
                    continue;
                if (level[edges[id].v] != -1)
                    continue;
                level[edges[id].v] = level[u] + 1;
                q.push(edges[id].v);
            }
        }
        return level[t] != -1;
    }
    ll dfs(int u, ll pushed) {
        if (pushed == 0)
            return 0;
        if (u == t)
            return pushed;
        for (int& cid = ptr[u]; cid < len(adj[u]); cid++) {
            int id = adj[u][cid];
            int v = edges[id].v;
            if (level[u] + 1 != level[v])
                continue;
            ll tr = dfs(v, min(pushed, edges[id].resid()));
            if (tr == 0)
                continue;
            edges[id].f += tr;
            edges[id ^ 1].f -= tr;
            return tr;
        }
        return 0;
    }
    ll flow() {
        ll f = 0;
        while (true) {
            fill(level.begin(), level.end(), -1);
            level[s] = 0;

```

```

            q.push(s);
            if (!bfs())
                break;
            fill(ptr.begin(), ptr.end(), 0);
            while (ll pushed = dfs(s, INF)) {
                f += pushed;
            }
        }
        return f;
    }
};

4.6.8 Ford-Fulkerson Directed

struct flow_network {
    vector<vector<ll>> f, c;
    vector<vector<int>> adj;
    vector<int> par;
    int n, s, t;

    flow_network(int n, int s, int t) : n(n), s(s), t(t) {
        adj.resize(n);
        par.resize(n);
        f.resize(n, vector<ll>(n, 0));
        c.resize(n, vector<ll>(n, 0));
    }
    void add_edge(int u, int v, ll cap) {
        adj[u].push_back(v);
        c[u][v] = cap;
        adj[v].push_back(u);
        c[v][u] = 0;
    }
    ll bfs() {
        fill(par.begin(), par.end(), -1);
        par[s] = -2;
        queue<pair<int, ll>> q;
        q.push({s, INF});

        while (!q.empty()) {
            int u = q.front().first;
            ll flow = q.front().second; q.pop();

            for (int v : adj[u]) {
                if (c[u][v]-f[u][v] > 0 and par[v] == -1) {
                    par[v] = u;
                    ll new_flow = min(flow, c[u][v]-f[u][v]);
                    if (v == t)
                        return new_flow;
                    q.push({v, new_flow});
                }
            }
        }
        return 0;
    }
    ll max_flow() {
        ll flow = 0;
        while (ll new_flow = bfs()) {
            flow += new_flow;
            int v = t;
            while (v != s) {
                int u = par[v];
                f[u][v] += new_flow;
                f[v][u] -= new_flow;
                v = u;
            }
        }
        return flow;
    }
};

4.6.9 Ford-Fulkerson Undirected
struct edge {

```

```

    size_t i;
    int v;
    ll c, f;
    ll resid() { return c - f; }
};

struct flow_network {
    vector<edge> edges;
    vector<vector<int>> adj;
    vector<int> par;
    int n, s, t;

    flow_network(int n, int s, int t) : n(n), s(s), t(t) {
        adj.resize(n);
        par.resize(n);
    }

    void add_edge(int u, int v, ll cap) {
        edges.push_back({edges.size(), v, cap, 0});
        adj[u].push_back(edges.size()-1);
        edges.push_back({edges.size(), u, 0, 0});
        adj[v].push_back(edges.size()-1);
    }

    bool find_aug_path() {
        fill(par.begin(), par.end(), -1);
        par[s] = -2;
        queue<int> q;
        q.push(s);
        while (!q.empty()) {
            int u = q.front(); q.pop();
            if (u == t)
                break;
            for (int i : adj[u]) {
                edge& e = edges[i];
                if (e.resid() > 0 && par[e.v] == -1) {
                    par[e.v] = e.i;
                    q.push(e.v);
                }
            }
        }
        return par[t] != -1;
    }
    ll augment() {
        ll flow = INF;
        for (int v = t; v != s; v = edges[par[v]^1].v)
            flow = min(flow, edges[par[v]].resid());
        for (int v = t; v != s; v = edges[par[v]^1].v) {
            edges[par[v]].f += flow;
            edges[par[v]^1].f -= flow;
        }
        return flow;
    }
    ll max_flow() {
        ll flow = 0;
        while (find_aug_path()) {
            flow += augment();
        }
        return flow;
    }
};

```

4.6.10 Kuhn

```

struct flow_network {
    int n;
    vector<vector<int>> adj;
    vector<int> mt, used;

    flow_network(int n) : n(n) {
        adj.resize(n);
        mt.resize(n, -1);
        used.resize(n, false);
    }
}

```

```

void add_edge(int u, int v) {
    adj[u].push_back(v);
    adj[v].push_back(u);
}

bool kuhn(int u) {
    if (used[u])
        return false;
    used[u] = true;
    for (int v : adj[u]) {
        if (mt[v] == -1 or kuhn(mt[v])) {
            mt[v] = v;
            return true;
        }
        return false;
    }
}

};

4.6.11 Min Cost Max Flow

struct edge {
    int u, v; ld cap, cost, flow;
    edge(int u, int v, ld cap, ld cost) : u(u), v(v), cap(cap),
    cost(cost), flow(0) {}
    ld res() {
        return cap - flow;
    }
};

struct flow_network {
    int n, s, t, *par, *in_queue, *num_vis;
    ld *cost, *pot;
    vector<edge> edges;
    vector<int> *adj;
    map<pair<int,int>, vector<int>> edge_idx;

    flow_network(int n, int s, int t) : n(n), s(s), t(t) {
        adj = new vector<int>[n];
        par = new int[n];
        in_queue = new int[n];
        num_vis = new int[n];
        cost = new ld[n];
        pot = new ld[n];
        for (int i = 0; i < n; i++) {
            pot[i] = 0;
        }
    }

    void add_edge(int u, int v, ld cap, ld cost) {
        adj[u].push_back(edges.size());
        edge_idx[{u, v}].push_back(edges.size());
        edges.push_back({u, v, cap, cost});

        adj[v].push_back(edges.size());
        edge_idx[{v, u}].push_back(edges.size());
        edges.push_back({v, u, 0, -cost});
    }

    ld get_flow(int u, int v) {
        ld flow = 0;
        for (int i : edge_idx[{u, v}])
            flow += edges[i].flow;
        return flow;
    }

    void bellman_ford() {
        for (int i = 0; i < n; i++)
            pot[i] = INF;
        pot[s] = 0;
        for (int it = 0; it < n-1; it++) {
            for (auto e : edges) {
                if (e.res() > 0)
                    pot[e.v] = min(pot[e.v], pot[e.u] + e.cost);
            }
        }
    }
}

```

```

bool spfa() {
    queue<int> q; q.push(s);
    while (not q.empty()) {
        int u = q.front(); q.pop();
        in_queue[u] = 0;
        if (num_vis[u]++ >= n) {
            cost[u] = -INF;
            return false;
        }
        for (int v : adj[u]) {
            edge e = edges[v];
            if (e.res() <= 0)
                continue;
            ld nd = cost[u] + e.cost + pot[u] - pot[e.v];
            if (cost[e.v] > nd) {
                cost[e.v] = nd;
                par[e.v] = v;
                if (not in_queue[e.v]) {
                    q.push(e.v);
                    in_queue[e.v] = true;
                }
            }
        }
    }
    return cost[t] != INF;
}

bool aug_path() {
    for (int i = 0; i < n; i++) {
        par[i] = -1;
        in_queue[i] = 0;
        num_vis[i] = 0;
        cost[i] = INF;
    }
    cost[s] = 0;
    in_queue[s] = true;
    return spfa();
}

pdd max_flow(bool do_bellman_ford=false) {
    ld total_cost = 0, total_flow = 0;
    if (do_bellman_ford)
        bellman_ford();
    while (aug_path()) {
        ld flow = INF;
        for (int i = par[t]; i != -1; i = par[edges[i].u])
            flow = min(flow, edges[i].res());
        for (int i = par[t]; i != -1; i = par[edges[i].u]) {
            edges[i].flow += flow;
            edges[i^1].flow -= flow;
        }
        total_cost += flow * (cost[t] + pot[t] - pot[s]);
        total_flow += flow;
        for (int i = 0; i < n; i++) {
            if (par[i] != -1)
                pot[i] += cost[i];
        }
    }
    return {total_cost, total_flow};
}

};
// When updating cost, make sure to update the back edge to use -cost

```

5 Math

5.1 Binary and Ternary Search

```

#pragma once
#include "../1-Core/01-template.hpp"

// T: O(log(n)), M: O(1)
template<typename F>
lng binSearch(lng ok, lng ng, F f) {
    while (abs(ok - ng) > 1) {

```

```
// T: O(1) or O(log(n)), M: O(1)
template<int MOD>
struct ModInt {
    // T: O(n^(1/2))
    static constexpr bool isPrime() {
        if (MOD <= 1) { return false; }
        if (MOD == 2 || MOD == 3) { return true; }
        if (MOD % 2 == 0 || MOD % 3 == 0) { return false; }
        for (lng i = 5; i * i <= MOD; i += 6) {
            if (MOD % i == 0 || MOD % (i + 2) == 0) { return false; }
        }
        return true;
    }
    static constexpr bool is_prime = ModInt::isPrime();

    int n;
    template<typename T = int>
    requires std::is_integral_v<T> || std::is_same_v<T, lll> ||
    std::is_same_v<T, ulll>
    constexpr ModInt(T N = 0) {
        if constexpr (std::is_signed_v<T> || std::is_same_v<T, lll>) {
            n = N % MOD; n += (n < 0) * MOD; }
        else { n = N % MOD; }
    }

    explicit constexpr operator int() const { return n; }
    static constexpr ModInt init(int N) { ModInt res; res.n = N; return res; }

    template<typename T>
    requires std::is_integral_v<T> || std::is_same_v<T, lll> ||
    std::is_same_v<T, ulll>
    constexpr ModInt &operator=(T N) {
```

```

        if constexpr (std::is_signed_v<T> || std::is_same_v<T, lll>) {
            n = N % MOD; n += (n < 0) * MOD; }
        else { n = N % MOD; }
        return *this; }

constexpr ModInt &operator++() { n++; n -= (n == MOD) * MOD; return
*this; }
constexpr ModInt &operator--() { n += (n == 0) * MOD; n--; return
*this; }
constexpr ModInt operator++(int) { ModInt res = *this; ++*this;
return res; }
constexpr ModInt operator--(int) { ModInt res = *this; --*this;
return res; }
constexpr ModInt &operator+=(ModInt o) { n += o.n; n -= (n >= MOD)
* MOD; return *this; }
constexpr ModInt &operator-=(ModInt o) { n -= o.n; n += (n < 0) *
MOD; return *this; }
constexpr ModInt &operator*=(ModInt o) { n = lng(n) * o.n % MOD;
return *this; }
constexpr ModInt &operator/=(ModInt o) { return *this *= inv(o); }

constexpr ModInt operator+() const { return *this; }
constexpr ModInt operator-() const { return init((n != 0) * (MOD -
n)); }
friend constexpr ModInt operator+(ModInt a, ModInt b) { a += b;
return a; }
friend constexpr ModInt operator-(ModInt a, ModInt b) { a -= b;
return a; }
friend constexpr ModInt operator*(ModInt a, ModInt b) { a *= b;
return a; }
friend constexpr ModInt operator/(ModInt a, ModInt b) { a /= b;
return a; }

// T: O(log(n))
friend constexpr ModInt inv(ModInt a) {
    assert(a != 0);
    if constexpr (is_prime) { return pow(a, MOD - 2); }
    int n = a.n, b = MOD, x = 1, y = 0;
    while (b > 0) {
        int q = n / b;
        n = std::exchange(b, n - q * b);
        x = std::exchange(y, x - q * y);
    }
    return init(x + (x < 0) * MOD); }

// T: O(log(n))
friend constexpr ModInt pow(ModInt a, lng b) {
    ulng ub = b < 0 ? -ulng(b) : ulng(b);
    if (b < 0) { a = inv(a); }
    ModInt res = 1;
    for (; ub > 0; ub >>= 1, a *= a) { if (ub & 1) { res *= a; } }
    return res; }

// T: O(log(n))
friend constexpr ModInt sqrt(ModInt a) {
    assert(is_prime);
    if (a == 0 || MOD == 2) { return a; }
    if (pow(a, (MOD - 1) / 2) != 1) { return init(-1); }
    ModInt b = 1;
    while (pow(b * b - a, (MOD - 1) / 2) == 1) { b++; }
    struct Node {
        ModInt x, y, w;
        constexpr Node(ModInt X = 0, ModInt Y = 0, ModInt W = 0) :
        x(X), y(Y), w(W) {}
        constexpr Node operator*(const Node &o) const {
            return Node(x * o.x + y * o.y * w, x * o.y + y * o.x,
            w); };
    };
    Node res(1, 0, b * b - a), cur(b, 1, b * b - a);
    for (int p = (MOD + 1) / 2; p > 0; p >>= 1, cur = cur * cur) {
        if (p & 1) { res = res * cur; }
    }
    return init(min(res.x.n, MOD - res.x.n)); }

explicit constexpr operator bool() const { return n != 0; }
constexpr bool operator!() const { return n == 0; }
friend constexpr auto operator<=>(const ModInt &a, const ModInt &b)
= default;

```

```

friend ostream &operator<<(ostream &is, ModInt &a) { lng b; is <<
b; a = ModInt(b); return is; }
friend ostream &operator<<(ostream &os, ModInt a) { return os <<
a.n; }
};
using mint = ModInt<998'244'353>;

#pragma once
#include "../1-Core/01-template.hpp"

// T: O(1) or O(log(n)), M: O(1)
template<int ID = 0>
struct DynModInt {
    static inline int MOD = 998'244'353;
    static inline ulng IMOD = -1ULL / 998'244'353 + 1;
    static inline bool is_prime = true;
    // T: O(n^(1/2))
    static bool isPrime() {
        if (MOD <= 1) { return false; }
        if (MOD == 2 || MOD == 3) { return true; }
        if (MOD % 2 == 0 || MOD % 3 == 0) { return false; }
        for (lng i = 5; i * i <= MOD; i += 6) {
            if (MOD % i == 0 || MOD % (i + 2) == 0) { return false; }
        }
        return true;
    }
    // T: O(n^(1/2))
    static void setMod(int mod) {
        assert(mod > 0);
        MOD = mod; IMOD = -1ULL / mod + 1; is_prime = isPrime();
    }
    static uint red(ulng a) {
        ulng q = ulng((ulll(a) * IMOD) >> 64);
        ulng r = a - q * MOD;
        if (MOD <= r) { r -= MOD; }
        return uint(r);
    }

    int n;
    template<typename T = int>
    requires std::is_integral_v<T> || std::is_same_v<T, lll> ||
    std::is_same_v<T, ulll>
    DynModInt(T N = 0) {
        if constexpr (std::is_signed_v<T> || std::is_same_v<T, lll>) {
            if (N >= 0) { n = red(N); }
            else { n = red(-ulng(N)); n = (n != 0) * (MOD - n); }
        }
        else { n = red(N); }
    }

    explicit operator int() const { return n; }
    static DynModInt init(int N) { DynModInt res; res.n = N; return
res; }
    template<typename T>
    requires std::is_integral_v<T> || std::is_same_v<T, lll> ||
    std::is_same_v<T, ulll>
    DynModInt &operator=(T N) {
        if constexpr (std::is_signed_v<T> || std::is_same_v<T, lll>) {
            if (N >= 0) { n = red(N); }
            else { n = red(-ulng(N)); n = (n != 0) * (MOD - n); }
        }
        else { n = red(N); }
        return *this;
    }

    DynModInt &operator++() { n++; n -= (n == MOD) * MOD; return *this;
    }
    DynModInt &operator--() { n += (n == 0) * MOD; n--; return *this; }
    DynModInt operator++(int) { DynModInt res = *this; ++*this; return
res; }
    DynModInt operator--(int) { DynModInt res = *this; --*this; return
res; }
    DynModInt &operator+=(DynModInt o) { n += o.n; n -= (n >= MOD) *
MOD; return *this; }
    DynModInt &operator-=(DynModInt o) { n -= o.n; n += (n < 0) * MOD;
return *this; }
    DynModInt &operator*=(DynModInt o) { n = red(1ULL * n * o.n);
return *this; }
    DynModInt &operator/=(DynModInt o) { return *this *= inv(o); }

```



```

return res;}
// T:  $O(n^3)$ 
friend Matrix inv(Matrix a)
requires (!std::is_integral_v<T>) && (requires { T::is_prime; } ||
std::is_floating_point_v<T>) {
    assert(a.n == a.m);
    return solveLin(std::move(a), eye(a.n, a.n));}
// T:  $O(n^3 * \log(n))$ 
friend Matrix pow(Matrix a, lng b) {
    assert(a.n == a.m && b >= 0);
    Matrix res = eye(a.n, a.n), tmp(a.n, a.n);
    while (b > 0) {
        if (b & 1) { res *= a; }
        b >>= 1;
        if (b > 0) { a *= a; }}
    return res;}
// T:  $O(n^3)$ 
friend Matrix solveLin(Matrix a, Matrix b)
requires (!std::is_integral_v<T>) && (requires { T::is_prime; } ||
std::is_floating_point_v<T>) {
    assert(a.n == b.n);
    int r = 0; vector<int> pivs(a.m, -1);
    for (int c = 0; r < a.n && c < a.m; c++) {
        int piv = r;
        if constexpr (std::is_floating_point_v<T>) {
            for (int p = r + 1; p < a.n; p++) {
                if (abs(a[p][c]) > abs(a[piv][c])) { piv = p; }}
            if (isNil(a[piv][c])) { piv = a.n; }}
        else { while (piv < a.n && isNil(a[piv][c])) { piv++; }}
        if (piv == a.n) { continue; }
        if (r != piv) {
            std::swap_ranges(a[r], a[r] + a.m, a[piv]);
            std::swap_ranges(b[r], b[r] + b.m, b[piv]);}
        T d = T(1) / a[r][c]; a[r][c] = T(1);
        for (int j = c + 1; j < a.m; j++) { a[r][j] *= d; }
        for (int j = 0; j < b.m; j++) { b[r][j] *= d; }
        for (int i = 0; i < a.n; i++) {
            if (i == r || isNil(a[i][c])) { continue; }
            T a_ic = a[i][c]; a[i][c] = T(0);
            #ifdef __AVX2__
            if constexpr ((requires { T::is_prime; } && sizeof(T)
== 4) || std::is_floating_point_v<T>) {
                FastGaussian::elimRow(a[i] + c + 1, a[r] + c + 1,
a_ic, a.m - c - 1);
                FastGaussian::elimRow(b[i], b[r], a_ic, b.m);}
            else
            #endif
            { for (int j = c + 1; j < a.m; j++) { a[i][j] -=
a[r][j] * a_ic; }
                for (int j = 0; j < b.m; j++) { b[i][j] -= b[r][j]
* a_ic; }}}
            pivs[c] = r++;}
        for (int i = r; i < a.n; i++) {
            for (int j = 0; j < b.m; j++) {
                if (!isNil(b[i][j])) { return Matrix(); }}}
    Matrix res(a.m, b.m);
    for (int c = 0; c < a.m; c++) {
        if (pivs[c] != -1) { std::copy_n(b[pivs[c]], b.m, res[c]); }
    }
    return res;}
// T:  $O(n)$ 
friend T trc(const Matrix &a) {
    assert(a.n == a.m);
    T res = 0;
    for (int i = 0; i < a.n; i++) { res += a[i][i]; }
    return res;}
friend Matrix trp(const Matrix &a) {
    Matrix res(a.m, a.n);
    for (int i = 0; i < a.m; i++) {
        for (int j = 0; j < a.n; j++) { res[i][j] = a[j][i]; }}
    return res;}
// T:  $O(n^3)$ 
friend Matrix adj(const Matrix a0)

```

```
requires (!std::is_integral_v<T>) && (requires { T::is_prime; } ||
std::is_floating_point_v<T>) {
    assert(a0.n == a0.m);
    int n = a0.n;
    if (n == 0) { return Matrix(); }
    if (n == 1) { return Matrix(vector<T>{T(1)}, 0); }
    Matrix a = a0, b = eye(n, n);
    int r = 0; T det_a = 1; vector<int> pivs(n, -1);
    for (int c = 0; c < n; c++) {
        int piv = r;
        if constexpr (std::is_floating_point_v<T>) {
            for (int p = r + 1; p < a.n; p++) {
                if (abs(a[p][c]) > abs(a[piv][c])) { piv = p; }
                if (isNil(a[piv][c])) { piv = a.n; }
            }
        } else { while (piv < a.n && isNil(a[piv][c])) { piv++; } }
        if (piv == n) { det_a = T(0); continue; }
        if (r != piv) {
            std::swap_ranges(a[r], a[r] + n, a[piv]);
            std::swap_ranges(b[r], b[r] + n, b[piv]);
            det_a = -det_a;
        }
        det_a *= a[r][c]; T d = T(1) / a[r][c]; a[r][c] = T(1);
        for (int j = c + 1; j < n; j++) { a[r][j] *= d; }
        for (int j = 0; j < n; j++) { b[r][j] *= d; }
        for (int i = 0; i < n; i++) {
            if (i == r || isNil(a[i][c])) { continue; }
            T a_ic = a[i][c]; a[i][c] = T(0);
            #ifdef __AVX2__
            if constexpr ((requires { T::is_prime; } && sizeof(T)
== 4) || std::is_floating_point_v<T>) {
                FastGaussian::elimRow(a[i] + c + 1, a[r] + c + 1,
a_ic, n - c - 1);
                FastGaussian::elimRow(b[i], b[r], a_ic, n);
            } else
            #endif
            { for (int j = c + 1; j < n; j++) { a[i][j] -=
a[r][j] * a_ic; }
                for (int j = 0; j < n; j++) { b[i][j] -= b[r][j] *
a_ic; }}
            pivs[c] = r++;
        }
        if (!isNil(det_a)) {
            Matrix res(n, n);
            for (int c = 0; c < n; c++) { std::copy_n(b[pivs[c]], n,
res[c]); }
            res *= det_a;
            return res;
        }
        if (r < n - 1) { return Matrix(n, n); }
        Matrix u = ker(a0), v = ker(trp(a0));
        int nz_i = -1, nz_j = -1;
        for (int i = 0; i < n; i++) { if (!isNil(u[0][i])) { nz_i = i;
break; } }
        for (int j = 0; j < n; j++) { if (!isNil(v[0][j])) { nz_j = j;
break; } }
        Matrix sub(n - 1, n - 1);
        for (int i = 0, sr = 0; i < n; i++) {
            if (i == nz_j) { continue; }
            for (int j = 0, sc = 0; j < n; j++) {
                if (j == nz_i) { continue; }
                sub[sr][sc++] = a0[i][j];
            }
            sr++;
        }
        T cof = det(sub);
        if ((nz_i + nz_j) & 1) { cof = -cof; }
        T q = cof / (u[0][nz_i] * v[0][nz_j]);
        Matrix res(n, n);
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) { res[i][j] = u[0][i] * v[0][j]
* q; }
            return res;
        }
        // T: O(n^3)
    } friend vector<T> charPoly(Matrix a)
requires (!std::is_integral_v<T>) && (requires { T::is_prime; } ||
std::is_floating_point_v<T>) {
```



```

        r++;}
    return a;}

// T: O(1)
static bool isNil(const T &x) {
    if constexpr (std::is_floating_point_v<T>) { return abs(x) <
        1e-9; }
    else { return x == T(0); }}
friend bool operator==(const Matrix &a, const Matrix &b) = default;

friend ostream &operator<<(ostream &os, const Matrix &a) {
    if (a.n == 0 || a.m == 0) { return os << "[]"; }
    os << "[";
    for (int i = 0; i < a.n; i++) {
        os << "[";
        for (int j = 0; j < a.m; j++) { os << a[i][j] << (j < a.m -
            1 ? ", " : ""); }
        os << "]" << (i < a.n - 1 ? ", " : "");}
    return os << "];"}
};

```

5.4 Modulo Pow

```

#pragma once
#include "../1-Core/01-template.hpp"
#include "../5-Mathematics/02-exgcd.hpp"

```

```

// T: O(log(n)), M: O(1)
lng modPow(lng a, lng b, lng mod = INF64) {
    a = (a % mod + mod) % mod;
    if (b < 0) {
        lng x, y;
        if (exGcd(a, mod, x, y) != 1) { return -1; }
        a = (x % mod + mod) % mod; b = -b;}
    lng res = 1;
    while (b > 0) {
        if (b & 1) { res = lll(res) * a % mod; }
        a = lll(a) * a % mod; b >>= 1;}
    return res;}

```

5.5 Modulo Combinatorics

```

struct ModFac {
    int n;
    vector<mint> fac, inv_fac;

    ModFac(int n): n(n) {
        fac.resize(n + 1);
        fac[0] = 1;
        for (int i = 1; i <= n; i++) {
            fac[i] = fac[i - 1] * i;
        }

        inv_fac.resize(n + 1);
        inv_fac[n] = fac[n].inv();
        for (int i = n - 1; i >= 0; i--) {
            inv_fac[i] = inv_fac[i + 1] * (i + 1);
        }
    }

    mint combiNR(int x, int y) const {
        if (y > x || y < 0) {
            return mint(0);
        }

        assert(x <= n);

        return fac[x] * inv_fac[y] * inv_fac[x - y];
    }
}

```

```

mint combiWR(int x, int y) const {
    if (x < 1 || y < 0) {
        return mint(0);
    }
}

```

```

    }

    assert(x + y - 1 <= n);

    return fac[x + y - 1] * inv_fac[y] * inv_fac[x - 1];
}

mint permuNR(int x, int y) const {
    if (y > x || y < 0) {
        return mint(0);
    }

    assert(x <= n);

    return fac[x] * inv_fac[x - y];
}

mint permuWR(int x, int y) const {
    if (y < 0) {
        return mint(0);
    }

    return mint(x).pow(y);
}
};

```

5.6 Dynamic MEX

```

// S: O(n * log(n)), U: O(log(n)), Q: O(1), M: O(n)
struct DynMex {
    int n; vector<int> a;
    map<int, int> frq; set<int> mis;

    DynMex(const vector<int> &A) : n(A.size()), a(A) {
        for (int i = 0; i <= n; i++) { mis.insert(i); }
        for (int x : a) { frq[x]++; mis.erase(x); }
    }

    int query() const { return *mis.begin(); }

    void update(int i, int v) {
        if (--frq[a[i]] == 0) { mis.insert(a[i]); }
        a[i] = v;
        frq[v]++; mis.erase(v);
    }
};

```

5.7 Extended Euclidean GCD

```

#pragma once
#include "../1-Core/01-template.hpp"

// T: O(log(n)), M: O(1)
lng exGcd(lng a, lng b, lng &x, lng &y) {
    x = 1; y = 0; lng X = 0, Y = 1;
    while (b != 0) {
        lng q = a / b;
        x = std::exchange(X, x - q * X);
        y = std::exchange(Y, y - q * Y);
        a = std::exchange(b, a % b);}
    return a;}

```

5.8 Sieve of Eratosthenes

```

#pragma once
#include "../1-Core/01-template.hpp"

// S: O(n * loglog(n)), U: NA, Q: O(1), M: O(n)
struct SieveOfErath {
    int n; vector<char> is_prime;
    SieveOfErath(int N) : n(N) {
        is_prime.assign(n + 1, true);
        if (n >= 0) { is_prime[0] = false; }
    }
}

```

```

        if (n >= 1) { is_prime[i] = false; }
        for (int i = 4; i <= n; i += 2) { is_prime[i] = false; }
        for (int i = 3; 1LL * i * i <= n; i += 2) {
            if (!is_prime[i]) { continue; }
            for (lng j = 1LL * i * i; j <= n; j += 2 * i) { is_prime[j]
                = false; }}
    };
}

```

5.9 Linear Sieve

```

#pragma once
#include "../1-Core/01-template.hpp"

// S: O(n), U: NA, Q: O(1), M: O(n)
struct LinearSieve {
    int n;
    vector<int> prms, spf, num_div, phi;
    vector<lng> sum_div; vector<signed char> mu;
    LinearSieve(int N) : n(N) {
        spf.assign(n + 1, -1);
        for (int i = 2; i <= n; i++) {
            if (spf[i] == -1) { prms.push_back(i); spf[i] = i; }
            for (int p : prms) {
                if (1LL * i * p > n) { break; }
                spf[i * p] = p;
                if (i % p == 0) { break; }}}}

    void getNumDiv() {
        num_div.assign(n + 1, 0); vector<int> spf_exp(n + 1, 0);
        if (n >= 1) { num_div[1] = 1; }
        for (int i = 2; i <= n; i++) {
            int p = spf[i], j = i / p;
            if (j % p == 0) {
                spf_exp[i] = spf_exp[j] + 1;
                num_div[i] = num_div[j] * (spf_exp[j] + 1) *
                    (spf_exp[i] + 1);}
            else {
                spf_exp[i] = 1;
                num_div[i] = num_div[j] * 2;}}}

    void getSumDiv() {
        sum_div.assign(n + 1, 0); vector<lng> spf_exp_sum(n + 1, 0);
        if (n >= 1) { sum_div[1] = 1; spf_exp_sum[1] = 1; }
        for (int i = 2; i <= n; i++) {
            int p = spf[i], j = i / p;
            if (j % p == 0) {
                spf_exp_sum[i] = spf_exp_sum[j] * p + 1;
                sum_div[i] = sum_div[j] * spf_exp_sum[j] *
                    spf_exp_sum[i];}
            else {
                spf_exp_sum[i] = p + 1;
                sum_div[i] = sum_div[j] * (p + 1);}}}

    void getPhi() {
        phi.assign(n + 1, 0);
        if (n >= 1) { phi[1] = 1; }
        for (int i = 2; i <= n; i++) {
            int p = spf[i], j = i / p;
            if (j % p == 0) { phi[i] = phi[j] * p; }
            else { phi[i] = phi[j] * (p - 1); }}}

    void getMu() {
        mu.assign(n + 1, 0);
        if (n >= 1) { mu[1] = 1; }
        for (int i = 2; i <= n; i++) {
            int p = spf[i], j = i / p;
            if (j % p == 0) { mu[i] = 0; }
            else { mu[i] = -mu[j]; }}}

    // Q: O(log(n))
    vector<pair<int, int>> getPrimeFac(int a) {
        assert(a <= n);
        vector<pair<int, int>> res;
        while (a > 1) {
            int p = spf[a], cnt = 0;
            while (spf[a] == p) { cnt++; a /= p; }
            res.push_back({p, cnt});}
        return res;}
}

```

```
};
```

5.10 Segmented Sieve

```
#pragma once
#include "../1-Core/01-template.hpp"

// S:  $O(r^{1/2} * \log\log(r^{1/2})) + n * \log\log(r)$ , U: NA, Q:  $O(1)$ , M:  $O(r^{1/2} + n)$ 
struct SegmentedSieve {
    lng l, r; vector<char> is_prime;
    vector<lng> bprms, prms, num_div, sum_div, phi, rem;
    vector<signed char> mu;
    SegmentedSieve(lng L, lng R) : l(L), r(R) {
        assert(0 <= l && l <= r);
        lng lim = sqrt(r) + 1;
        while (lim * lim > r) { lim--; }
        vector<char> is_bprime(lim + 1, true);
        if (lim >= 0) { is_bprime[0] = false; }
        if (lim >= 1) { is_bprime[1] = false; }
        if (lim >= 2) { bprms.push_back(2); }
        for (int i = 4; i <= lim; i += 2) { is_bprime[i] = false; }
        for (lng i = 3; i <= lim; i += 2) {
            if (!is_bprime[i]) { continue; }
            bprms.push_back(i);
            for (lng j = i * i; j <= lim; j += 2 * i) { is_bprime[j] = false; }
        }
        is_prime.assign(r - l + 1, true);
        if (l <= 1) { is_prime[0] = false; if (l == 0 && r >= 1) { is_prime[1] = false; } }
        for (lng p : bprms) {
            lng j0 = max(p * p, (l + p - 1) / p * p);
            for (lng j = j0; j <= r; j += p) { is_prime[j - l] = false; }
        }
        for (lng i = 0; i <= r - l; i++) {
            if (is_prime[i]) { prms.push_back(i + l); }
        }
    }

    void getNumDiv() {
        num_div.assign(r - l + 1, 1); rem.resize(r - l + 1);
        for (lng i = 0; i <= r - l; i++) { rem[i] = i + l; }
        if (l == 0) { num_div[0] = 0; rem[0] = 1; }
        for (lng p : bprms) {
            lng j0 = (l + p - 1) / p * p;
            if (j0 == 0) { j0 = p; }
            for (lng j = j0; j <= r; j += p) {
                lng cnt = 0;
                while (rem[j - l] % p == 0) { rem[j - l] /= p; cnt++; }
                num_div[j - l] *= (cnt + 1); }
        }
        for (lng i = 0; i <= r - l; i++) {
            if (rem[i] > 1) { num_div[i] *= 2; }
        }
    }

    void getSumDiv() {
        sum_div.assign(r - l + 1, 1); rem.resize(r - l + 1);
        for (lng i = 0; i <= r - l; i++) { rem[i] = i + l; }
        if (l == 0) { sum_div[0] = 0; rem[0] = 1; }
        for (lng p : bprms) {
            lng j0 = (l + p - 1) / p * p;
            if (j0 == 0) { j0 = p; }
            for (lng j = j0; j <= r; j += p) {
                lng cur = 1, sum = 1;
                while (rem[j - l] % p == 0) { rem[j - l] /= p; cur *= p; sum += cur; }
                sum_div[j - l] *= sum; }
        }
        for (lng i = 0; i <= r - l; i++) {
            if (rem[i] > 1) { sum_div[i] *= (rem[i] + 1); }
        }
    }

    void getPhi() {
        phi.assign(r - l + 1, 0); rem.resize(r - l + 1);
        for (lng i = 0; i <= r - l; i++) { phi[i] = rem[i] = i + l; }
        if (l == 0) { phi[0] = 0; rem[0] = 1; }
        for (lng p : bprms) {
            lng j0 = (l + p - 1) / p * p;
            if (j0 == 0) { j0 = p; }
            for (lng j = j0; j <= r; j += p) {
                phi[j - l] -= phi[j - l] / p;
                while (rem[j - l] % p == 0) { rem[j - l] /= p; }
            }
        }
    }
};
```

```
for (lng i = 0; i <= r - l; i++) {
    if (rem[i] > 1) { phi[i] -= phi[i] / rem[i]; }
}

void getMu() {
    mu.assign(r - l + 1, 1); rem.resize(r - l + 1);
    for (lng i = 0; i <= r - l; i++) { rem[i] = i + l; }
    if (l == 0) { mu[0] = 0; rem[0] = 1; }
    for (lng p : bprms) {
        lng j0 = (l + p - 1) / p * p;
        if (j0 == 0) { j0 = p; }
        for (lng j = j0; j <= r; j += p) {
            lng cnt = 0;
            while (rem[j - l] % p == 0) { rem[j - l] /= p; cnt++; }
            if (cnt > 1) { mu[j - l] = 0; }
            else { mu[j - l] = -mu[j - l]; }
        }
    }
    for (lng i = 0; i <= r - l; i++) {
        if (rem[i] > 1) { mu[i] = -mu[i]; }
    }
};
```

5.11 Is Prime Miller-Rabin

```
#pragma once
#include "../1-Core/01-template.hpp"

// T:  $O(\log(n))$ , M:  $O(1)$ 
bool isPrimeMR(lng n) { // Miller-Rabin
    if (n < 2) { return false; }
    if (n == 2 || n == 3) { return true; }
    if (n % 2 == 0) { return false; }
    int s = __builtin_ctzll(n - 1); lng d = (n - 1) >> s;
    static constexpr lng bsm[] = {2LL, 7LL, 61LL};
    static constexpr lng blg[] = {2LL, 325LL, 9375LL, 28178LL, 450775LL, 9780504LL, 1795265022LL};
    const lng *B = n < 4759123141LL ? bsm : blg;
    for (int i = 0; i < (n < 4759123141LL ? 3 : 7); i++) {
        lng a = B[i];
        if (a % n == 0) { continue; }
        a %= n; lng b = d, x = 1;
        while (b > 0) {
            if (b & 1) { x = ulll(x) * a % n; }
            a = ulll(a) * a % n;
            b >>= 1; }
        if (x == 1 || x == n - 1) { continue; }
        bool is_comp = true;
        for (int j = 1; j < s; j++) {
            x = ulll(x) * x % n;
            if (x == n - 1) { is_comp = false; break; }
            if (x == 1) { return false; }
        }
        if (is_comp) { return false; }
    }
    return true;
}
```

5.12 Get One Factor Brent's Pollard-Rho

```
#pragma once
#include "../1-Core/01-template.hpp"

// T:  $O(n^{1/4})$ , M:  $O(1)$ 
lng getOneFacBPR(lng n, lng x0 = 2, lng c = 1) { // Brent's Pollard's Rho
    auto f = [](lng x, lng c, lng mod) -> lng {
        return (ulll(x) * x + c) % mod;
    };
    int m = 128, l = 1;
    lng x = x0, xs = -1, y = -1, g = 1, q = 1;
    while (g == 1) {
        y = x; int k = 0;
        while (g == 1 && k < l) {
            xs = x;
            for (int i = 0; i < min(m, l - k); i++) {
                x = f(x, c, n);
                q = ulll(q) * (x > y ? x - y : y - x) % n;
                g = gcd(q, n); k += m; }
            l *= 2; }
        if (g == n) {
            do {
```

```
xs = f(xs, c, n);
g = gcd(xs > y ? xs - y : y - xs, n); }
while (g == 1); }
return g; }
```

5.13 Get Prime Factors

```
#pragma once
#include "../1-Core/01-template.hpp"

// T:  $O(n^{1/2})$ , M:  $O(\log(n))$ 
vector<lng> getPrimeFacSlow(lng n) {
    vector<lng> res;
    for (int p : {2, 3, 5}) { while (n % p == 0) { res.push_back(p); n /= p; } }
    int j = 0; static constexpr int inc[] = {4, 2, 4, 2, 4, 6, 2, 6};
    for (lng i = 7; i * i <= n; i += inc[j], j = (j + 1) & 7) {
        while (n % i == 0) { res.push_back(i); n /= i; }
    }
    if (n > 1) { res.push_back(n); }
    sort(res.begin(), res.end());
    return res; }

#pragma once
#include "../1-Core/01-template.hpp"
#include "../5-Mathematics/05-isprimemr.hpp"
#include "../5-Mathematics/07-getonefacbpr.hpp"
#include "../6-Miscellaneous/random.hpp"

// T:  $O(n^{1/4})$ , M:  $O(\log(n))$ 
Random rnd;
void getPrimeFacFast(lng n, vector<lng> &res) {
    if (n <= 1) { return; }
    if (isPrimeMR(n)) { res.push_back(n); return; }
    static constexpr int prms[] = {2, 3, 5, 7, 11, 13, 17, 19, 23, 29};
    for (int p : prms) {
        if (n % p == 0) {
            res.push_back(p);
            getPrimeFacFast(n / p, res);
            return; }
    }
    lng div = n, x0 = rnd.randLng(2, n - 1), c = rnd.randLng(1, n - 1);
    while (div == n) {
        div = getOneFacBPR(n, x0, c);
        if (div == n) {
            x0 = rnd.randLng(2, n - 1);
            c = rnd.randLng(1, n - 1); }
    }
    getPrimeFacFast(div, res);
    getPrimeFacFast(n / div, res); }

#pragma once
#include "../1-Core/01-template.hpp"
#include "../5-Mathematics/06-getprimefacslow.hpp"
#include "../5-Mathematics/08-getprimefacfast.hpp"

// T:  $O(n^{1/4})$  or  $O(n^{1/2})$ , M:  $O(d(n))$ 
vector<lng> getAllFac(lng n, bool opt = 0) {
    if (n <= 0) { return {}; }
    if (n == 1) { return {1}; }
    vector<lng> prms;
    /**
    if (!opt) { prms = getPrimeFacSlow(n); } /**/
    /**
    if (opt) { getPrimeFacFast(n, prms); } /**/
    sort(prms.begin(), prms.end());
    vector<lng> res = {1};
    for (int i = 0; i < prms.size(); i++) {
        int cnt = 0; lng p = prms[i];
        while (i < prms.size() && prms[i] == p) { cnt++; i++; }
        int len = res.size();
        for (int j = 0; j < len; j++) {
            lng cur = res[j];
            for (int k = 0; k < cnt; k++) { cur *= p; }
            res.push_back(cur); }
    }
```

5.14 Get All Factors

```
// T:  $O(n^{1/4} * \log(n))$  or  $O(n^{1/2} * \log(n))$ , M:  $O(\log(n))$ 
lng getPrimRoot(lng mod, bool opt = 0) {
```

```

if (mod <= 0) { return -1; }
if (mod == 1 || mod == 2 || mod == 4) { return mod - 1; }
if (mod % 4 == 0) { return -1; }
vector<lng> prms;
/**
if (!opt) { prms = getPrimeFacSlow(mod >> (!mod & 1)); }/**/
if (opt) { getPrimeFacFast(mod >> (!mod & 1), prms); }/**/
sort(prms.begin(), prms.end());
if (!prms.empty()) { if (prms[0] != prms.back()) { return -1; } }
lng phi = getPhi(mod, opt); prms.clear();
/**
if (!opt) { prms = getPrimeFacSlow(phi); }/**/
if (opt) { getPrimeFacFast(phi, prms); }/**/
sort(prms.begin(), prms.end());
prms.erase(unique(prms.begin(), prms.end()), prms.end());
for (lng res = 2; res < mod; res++) {
    if (gcd(res, mod) != 1) { continue; }
    bool chk = true;
    for (int i = 0; i < prms.size() && chk; i++) {
        if (modPow(res, phi / prms[i], mod) == 1) { chk = false; }
    }
    if (chk) { return res; }
}
return -1;
}

```

5.23 Modulo Root

```

#pragma once
#include "../1-Core/01-template.hpp"
#include "../5-Mathematics/02-eagcd.hpp"
#include "../5-Mathematics/10-getphi.hpp"
#include "../5-Mathematics/16-modlog.hpp"
#include "../5-Mathematics/17-getprimroot.hpp"

```

```

// T:  $O(n^{1/2})$ , M:  $O(n^{1/2})$ 
vector<lng> modRoot(lng a, lng b, lng mod, bool opt = 0) { //  $x^a = b \pmod m$ 
    if (b == 0) { return {0}; }
    lng g = getPrimRoot(mod, opt);
    if (g == -1) { return {}; }
    lng k = modLog(g, b, mod);
    if (k == -1) { return {}; }
    lng phi = getPhi(mod, opt), x, y, d = exGcd(a, phi, y, x);
    if (k % d != 0) { return {}; }
    y = (lll(y) % phi + phi) % phi * (k / d) % phi;
    vector<lng> res;
    for (lng i = 0; i < d; i++) {
        res.push_back(modPow(g, y + i * (phi / d), mod));
    }
    sort(res.begin(), res.end());
    res.erase(unique(res.begin(), res.end()), res.end());
    return res;
}

```

5.24 Floor Sum

```

#pragma once
#include "../1-Core/01-template.hpp"

```

```

// T:  $O(\log(n))$ , M:  $O(1)$ 
lll getFloorSum(lng n, lng a, lng b, lng mod) { //  $\sum_{i=0}^{n-1} \text{floor}((a * x + b) / m)$ 
    lll res = 0;
    if (a >= mod) { res += lll(n - 1) * n / 2 * (a / mod); a %= mod; }
    if (b >= mod) { res += lll(n) * (b / mod); b %= mod; }
    lng y = (lll(a) * n + b) / mod;
    if (y == 0) { return res; }
    lll x = lll(y) * mod - b;
    res += lll(n - (x + a - 1) / a) * y;
    res += getFloorSum(y, mod, (a - x % a) % a, a);
    return res;
}

```

5.25 Montgomery Multiplication

```

#pragma once

```

```

#include "../1-Core/01-template.hpp"
// S:  $O(1)$ , U: NA, Q:  $O(1)$ , M:  $O(1)$ 
struct Montgomery {
    ulng mod, inv, rsq;
    Montgomery(ulng n = 1) : mod(n), inv(n) {
        assert(n & 1);
        for (int i = 0; i < 5; i++) { inv *= 2 - mod * inv; }
        inv = -inv; rsq = -ulll(mod) % mod; }

    ulng red(ulll x) const {
        ulng q = ulng(x) * inv, res = (x + ulll(q) * mod) >> 64;
        return res - (res >= mod) * mod; }
    ulng init(ulng x) const { return red(ulll(x) * rsq); }
    ulng mul(ulng a, ulng b) const { return red(ulll(a) * b); }
    // Q:  $O(\log(n))$ 
    ulng pow(ulng a, ulng b) const {
        a = init(a); ulng res = init(1);
        while (b > 0) {
            if (b & 1) { res = mul(res, a); }
            b >>= 1;
            if (b > 0) { a = mul(a, a); }
        }
        return red(res); }
};

```

5.26 Poly FFT

```

#pragma once
#include "../1-Core/01-template.hpp"

```

```

// T:  $O(n)$  or  $O(n * \log(n))$ , M:  $O(n)$ 
struct Poly {
    using cld = std::complex<long double>;
    static constexpr long double pi = 3.14159265358979323846;

    vector<lng> v; // ascending powers
    Poly(int n = 1) : v(max(n, 1), 0) {}
    Poly(const vector<lng> &a) : v(a) { trim(); }
    Poly(vector<lng> &a) noexcept : v(std::move(a)) { trim(); }

    lng &operator[](int i) { return v[i]; }
    lng operator[](int i) const { return v[i]; }
    int deg() const { return v.size() - 1; }
    lng lead() const { return v.empty() ? 0 : v.back(); }
    int size() const { return v.size(); }
    void trim() {
        while (v.size() > 1 && v.back() == 0) { v.pop_back(); }
        if (v.empty()) { v.push_back(0); }
    }

    Poly &operator+=(const Poly &o) {
        if (o.size() > size()) { v.resize(o.size(), 0); }
        for (int i = 0; i < o.size(); i++) { v[i] += o[i]; }
        trim();
        return *this; }
    Poly &operator-=(const Poly &o) {
        if (o.size() > size()) { v.resize(o.size(), 0); }
        for (int i = 0; i < o.size(); i++) { v[i] -= o[i]; }
        trim();
        return *this; }
    Poly &operator*=(lng o) {
        if (o == 0) { v = {0}; return *this; }
        for (lng &x : v) { x *= o; }
        trim();
        return *this; }
    static void fft(vector<cld> &a, bool is_inv) {
        int n = a.size();
        for (int i = 1, j = 0; i < n; i++) {
            int bit = n >> 1;
            for (; j & bit; bit >>= 1) { j ^= bit; }
            if (i < j) { std::swap(a[i], a[j]); }
        }
        static vector<cld> w = {cld(0, 0), cld(1, 0)};
        if (w.size() < n) {
            int m = w.size(); w.resize(n);

```

```

            for (int len = m; len < n; len <= 1) {
                for (int i = 0; i < len; i++) {
                    w[len + i] = cld(std::cos(pi * i / len),
                        std::sin(pi * i / len)); }
            }
            for (int len = 1; len < n; len <= 1) {
                for (int i = 0; i < n; i += 2 * len) {
                    for (int j = 0; j < len; j++) {
                        cld u = a[i + j];
                        cld v = a[i + j + len] * w[len + j];
                        a[i + j] = u + v;
                        a[i + j + len] = u - v; }
            }
            if (is_inv) {
                reverse(a.begin() + 1, a.end());
                long double n_inv = 1.0L / n;
                for (cld &x : a) { x *= n_inv; }
            }
            Poly &operator+=(const Poly &o) {
                if (isNil() || o.isNil()) { v = {0}; return *this; }
                if (size() == 1) {
                    lng v0 = v[0]; v = o.v;
                    for (auto &x : v) { x *= v0; }
                    trim();
                    return *this; }
                if (o.size() == 1) {
                    for (lng &x : v) { x *= o[0]; }
                    trim();
                    return *this; }
                int n = 1;
                while (n < size() + o.size() - 1) { n <= 1; }
                #ifdef __AVX2__
                static vector<double> Ra, Ia, Rb, Ib;
                Ra.assign(v.begin(), v.end()); Ra.resize(n, 0);
                Ia.assign(n, 0);
                Rb.assign(o.v.begin(), o.v.end()); Rb.resize(n, 0);
                Ib.assign(n, 0);
                FastPoly::fft(Ra, Ia, false); FastPoly::fft(Rb, Ib, false);
                int i = 0;
                for (; i + 3 < n; i += 4) {
                    __m256d ra = __m256_loadu_pd(&Ra[i]), ia =
                        __m256_loadu_pd(&Ia[i]);
                    __m256d rb = __m256_loadu_pd(&Rb[i]), ib =
                        __m256_loadu_pd(&Ib[i]);
                    __m256d tr = __m256_sub_pd(__m256_mul_pd(ra, rb),
                        __m256_mul_pd(ia, ib));
                    __m256d ti = __m256_add_pd(__m256_mul_pd(ra, ib),
                        __m256_mul_pd(ia, rb));
                    __m256_storeu_pd(&Ra[i], tr); __m256_storeu_pd(&Ia[i],
                        ti);
                }
                for (; i < n; i++) {
                    double tr = Ra[i] * Rb[i] - Ia[i] * Ib[i];
                    double ti = Ra[i] * Ib[i] + Ia[i] * Rb[i];
                    Ra[i] = tr; Ia[i] = ti;
                }
                FastPoly::fft(Ra, Ia, true); v.resize(n);
                for (int i = 0; i < n; i++) { v[i] = std::llround(Ra[i]); }
                #else
                static vector<cld> fa, fb;
                fa.assign(v.begin(), v.end()); fa.resize(n);
                fb.assign(o.v.begin(), o.v.end()); fb.resize(n);
                fft(fa, false); fft(fb, false);
                for (int i = 0; i < n; i++) { fa[i] *= fb[i]; }
                fft(fa, true); v.resize(n);
                for (int i = 0; i < n; i++) { v[i] =
                    std::llround(fa[i].real()); }
                #endif
                trim();
                return *this; }
            }

```

```

Poly operator+(const Poly &a, const Poly &b) { return *this; }
Poly operator-(const Poly &a, const Poly &b) { return *this; }
Poly operator*(const Poly &a, const Poly &b) { return *this; }
Poly operator/=(const Poly &a, const Poly &b) { return *this; }
Poly operator+=(const Poly &a, const Poly &b) { return *this; }
Poly operator-=(const Poly &a, const Poly &b) { return *this; }
Poly operator*=(const Poly &a, const Poly &b) { return *this; }
Poly operator/=(const Poly &a, const Poly &b) { return *this; }

```

```

return *this;};

Moly &operator*=(mint o) {
    if (o.n == 0) { v = {mint::init(0)}; return *this; }
    for (mint &x : v) { x *= o; }
    trim();
    return *this;};

static void ntt(vector<mint> &a, bool is_inv) {
    #ifdef __AVX2__
    FastMoly::ntt(a, is_inv);
    #else
    int n = a.size();
    for (int i = 1, j = 0; i < n; i++) {
        int bit = n >> 1;
        for (; j < bit; bit >= 1) { j ^= bit; }
        j ^= bit;
        if (i < j) { std::swap(a[i], a[j]); }
    }
    static vector<mint> w = {mint::init(0), mint::init(1)};
    if (w.size() < n) {
        int m = w.size(); w.resize(n);
        for (int len = m; len < n; len <= 1) {
            mint w0 = pow(mint(NTT_G), (NTT_P - 1) / (2 * len));
            for (int i = len / 2; i < len; i++) {
                w[2 * i] = w[i]; w[2 * i + 1] = w[i] * w0; }
        }
        for (int len = 1; len < n; len <= 1) {
            for (int i = 0; i < n; i += 2 * len) {
                for (int j = 0; j < len; j++) {
                    uint u = a[i + j].n;
                    uint v = 1ULL * a[i + j + len].n * w[j + len].n %
                        NTT_P;
                    a[i + j].n = u + v - (u + v >= NTT_P) *
                        NTT_P;
                    a[i + j + len].n = u - v + (u < v) * NTT_P; }
            }
        }
        if (is_inv) {
            reverse(a.begin() + 1, a.end());
            mint n_inv = inv(mint(n));
            for (mint &x : a) { x *= n_inv; }
        }
    }
    #endif
}

Moly &operator*=(const Moly &o) {
    if (isNil() || o.isNil()) { v = {mint::init(0)}; return *this; }
    if (size() == 1) {
        mint v0 = v[0]; v = o.v;
        for (auto &x : v) { x *= v0; }
        trim();
        return *this;};
    if (o.size() == 1) {
        for (mint &x : v) { x *= o[0]; }
        trim();
        return *this;};
    int n = 1;
    while (n < size() + o.size() - 1) { n <= 1; }
    static vector<mint> fa, fb;
    fa.assign(v.begin(), v.end()); fa.resize(n, mint::init(0));
    fb.assign(o.v.begin(), o.v.end()); fb.resize(n, mint::init(0));
    ntt(fa, false); ntt(fb, false);
    for (int i = 0; i < n; i++) { fa[i] *= fb[i]; }
    ntt(fa, true);
    v.assign(fa.begin(), fa.begin() + size() + o.size() - 1);
    trim();
    return *this;};

friend pair<Moly, Moly> divMod(Moly a, const Moly &b) {
    if (a.size() < b.size()) { return {Moly({mint::init(0)}),
        std::move(a)}; }
    int len = a.size() - b.size() + 1; Moly a_rev(len), b_rev(len);
    for (int i = 0; i < min(len, a.size()); i++) { a_rev[i] =
        a[a.size() - i - 1]; }
    for (int i = 0; i < min(len, b.size()); i++) { b_rev[i] =
        b[b.size() - i - 1]; }
    Moly q = std::move(a_rev) * invS(b_rev, len - 1);
    q.v.resize(len); reverse(q.v.begin(), q.v.end()); q.trim();
    Moly r = std::move(a); r -= q * b; r.trim();
    return {std::move(q), std::move(r)};};

Moly &operator/=(const Moly &o) {

```

```

    if (size() < o.size()) { v = {mint::init(0)}; return *this; }
    return *this = divMod(std::move(*this), o).first; }

Moly &operator%=(const Moly &o) {
    if (size() < o.size()) { return *this; }
    return *this = divMod(std::move(*this), o).second; }

friend Moly operator+(Moly a) { return a; }
friend Moly operator-(Moly a) { for (mint &x : a.v) { x = -x; }
return a; }
friend Moly operator+(Moly a, const Moly &b) { a += b; return a; }
friend Moly operator-(Moly a, const Moly &b) { a -= b; return a; }
friend Moly operator*(Moly a, mint b) { a *= b; return a; }
friend Moly operator*(mint b, Moly a) { a *= b; return a; }
friend Moly operator*(Moly a, const Moly &b) { a *= b; return a; }
friend Moly operator/(Moly a, const Moly &b) { a /= b; return a; }
friend Moly operator%(Moly a, const Moly &b) { a %= b; return a; }

// Polynomial Ring Operations
friend mint eval(const Moly &a, mint c) { // A(c)
    uint res = 0;
    for (int i = a.size() - 1; i >= 0; i--) { res = (1ULL * res *
c.n + a[i].n) % NTT_P; }
    return mint::init(res); }
friend vector<mint> chirpZ(const Moly &a, mint c, mint r, int m) {
// A(c * r^-i), i in [0, m)
    if (a.isNil() || m <= 0) { return vector<mint>(max(m, 0),
mint::init(0)); }
    if (r == mint::init(0)) { vector<mint> res(m, a[0]); res[0] =
eval(a, c); return res; }
    int n = a.size();
    mint c_pow = mint::init(1), r_cur = mint::init(1), r_pow =
mint::init(1);
    mint r_inv = inv(r), r_icur = mint::init(1), r_ipow =
mint::init(1);
    Moly A(n), B(n + m - 1);
    for (int i = 0; i < n; i++) {
        A.v[n - i - 1] = a[i] * c_pow * r_ipow; c_pow *= c;
        r_ipow *= r_icur; r_icur *= r_inv; }
    r_cur = mint::init(1), r_pow = mint::init(1);
    for (int i = 0; i < n + m - 1; i++) { B.v[i] = r_pow; r_pow *=
r_cur; r_cur *= r; }
    A *= B;
    if (A.size() < n + m - 1) { A.v.resize(n + m - 1,
mint::init(0)); }
    A.v.erase(A.v.begin(), A.v.begin() + n - 1); A.v.resize(m);
    r_icur = mint::init(1), r_ipow = mint::init(1);
    for (int i = 0; i < m; i++) {
        A.v[i] *= r_ipow;
        r_ipow *= r_icur; r_icur *= r_inv; }
    return std::move(A.v); }
// T: O(n * log(n)^2)
friend vector<mint> eval(const Moly &a, const vector<mint> &c) { //
A(c_n), i in [0, m)
    if (c.empty()) { return {}; }
    int m = c.size(); vector<Moly> segt(4 * m);
    auto build = [&](auto &&build, int i, int l, int r) -> void {
        if (l == r) { segt[i] = Moly{c[l], mint::init(1)};
return; }
        int md = std::midpoint(l, r);
        build(build, 2 * i, l, md);
        build(build, 2 * i + 1, md + 1, r);
        segt[i] = segt[2 * i] * segt[2 * i + 1]; }
    build(build, 1, 0, m - 1);
    vector<mint> res(m);
    auto solve = [&](auto &&solve, int i, int l, int r, Moly p) ->
void {
        p %= segt[i];
        if (p.isNil()) { for (int j = l; j <= r; j++) { res[j] =
mint::init(0); } return; }
        if (l == r) { res[l] = p[0]; return; }
        int md = std::midpoint(l, r);
        solve(solve, 2 * i, l, md, p);
        solve(solve, 2 * i + 1, md, p);
    }
    solve(solve, 1, 0, m - 1, A);
    return res; }

```

```

        fac.push_back(fac.back() * mint::init(i));
        ifc.push_back(ifc.back() * inv.back());
        mint b_pow = mint::init(1); Moly A(n), B(n);
        for (int i = 0; i < n; i++) {
            A[n - i - 1] = a[i] * fac[i];
            B[i] = b_pow * ifc[i]; b_pow *= b;
        }
        A *= B;
        A.v.resize(n, mint::init(0));
        std::reverse(A.v.begin(), A.v.end());
        for (int i = 0; i < n; i++) { A[i] *= ifc[i]; }
        A.trim();
        return A;
    }
// T:  $O(n * \log(n)^2)$ 
static Moly inter(const vector<mint> &x, const vector<mint> &y) {
// Lagrange Interpolation
    assert(x.size() == y.size());
    if (x.empty()) { return Moly({mint::init(0)}); }
    int n = x.size(); vector<Moly> segt(4 * n);
    auto build = [&](auto &&build, int i, int l, int r) -> void {
        if (l == r) { segt[i] = Moly({-x[l], mint::init(1)});
            return; }
        int md = std::midpoint(l, r);
        build(build, 2 * i, l, md);
        build(build, 2 * i + 1, md + 1, r);
        segt[i] = segt[2 * i] * segt[2 * i + 1];
    };
    build(build, 1, 0, n - 1);
    vector<mint> dM(n);
    auto calc = [&](auto &&calc, int i, int l, int r, Moly p) ->
    void {
        p %= segt[i];
        if (p.isNil()) { for (int j = 1; j <= r; j++) { dM[j] =
            mint::init(0); } return; }
        if (l == r) { dM[l] = p[0]; return; }
        int md = std::midpoint(l, r);
        calc(calc, 2 * i, l, md, p);
        calc(calc, 2 * i + 1, md + 1, r, std::move(p));
        calc(calc, 1, 0, n - 1, deriv(segt[1]));
        auto solve = [&](auto &&solve, int i, int l, int r) -> Moly {
            if (l == r) { return Moly({y[l] * inv(dM[l])}); }
            int md = std::midpoint(l, r);
            Moly l_res = solve(solve, 2 * i, l, md);
            Moly r_res = solve(solve, 2 * i + 1, md + 1, r);
            l_res *= segt[2 * i + 1]; r_res *= segt[2 * i]; l_res +=
            r_res;
            return l_res;
        };
        return solve(solve, 1, 0, n - 1);
    }
}

// Formal Power Series Operations
// T:  $O(n^2 + n^{\frac{3}{2}} * \log(n))$ 
friend Moly comp(const Moly &a, const Moly &b, int deg) { //
    Brent-Kung:  $A(B(x)) \bmod x^{\deg+1}$ 
    if (a.isNil()) { return Moly({mint::init(0)}); }
    int n = min(a.size(), deg + 1), m = std::sqrt(n + 1);
    vector<Moly> b_pow(m + 1); b_pow[0] = Moly({mint::init(1)});
    for (int i = 1; i <= m; i++) {
        b_pow[i] = b_pow[i - 1] * b;
        if (b_pow[i].size() > deg + 1) { b_pow[i].v.resize(deg +
            1); }
        b_pow[i].trim();
    }
    Moly res, cur;
    for (int i = (n - 1) / m; i >= 0; i--) {
        cur.v.clear();
        for (int j = 0; j <= min(n - i * m - 1, m - 1); j++) {
            if (a[i * m + j].n == 0) { continue; }
            if (cur.size() < b_pow[j].size()) {
                cur.v.resize(b_pow[j].size(), mint::init(0));
                for (int k = 0; k < b_pow[j].size(); k++) {
                    cur[k] += b_pow[j][k] * a[i * m + j];
                }
            }
            if (!res.isNil()) {
                res = std::move(res) * b_pow[j];
                if (res.size() > deg + 1) { res.v.resize(deg + 1); }
            }
            if (res.size() < cur.size()) { res.v.resize(cur.size(),
                mint::init(0)); }
        }
    }
}

```

```

for (int j = 0; j < cur.size(); j++) { res[j] += cur[j];
    res.trim();}
if (res.size() > deg + 1) { res.v.resize(deg + 1);
    res.trim();
    return res;}
friend Moly deriv(Moly a) {
    if (a.size() <= 1) { return Moly({mint::init(0)}); }
    for (int i = 1; i < a.size(); i++) { a[i - 1] = a[i] *
        mint::init(i); }
    a.v.pop_back(); a.trim();
    return a;}
friend Moly exp(const Moly &a, int deg) { //  $e^A(x) \bmod x^{\deg + 1}$ 
    assert(a.isNil() || a[0].n == 0);
    Moly res({mint::init(1)});
    for (int len = 2; len < 2 * deg + 2; len <= 1) {
        Moly cur = log(res, len - 1); cur.v.resize(len,
            mint::init(0));
        for (int i = 0; i < len; i++) {
            cur[i] = (i < a.size() ? a[i] : mint::init(0)) -
                cur[i];}
        cur[0] += mint::init(1);
        res *= cur; res.v.resize(len, mint::init(0)); res.trim();}
    res.v.resize(deg + 1, mint::init(0)); res.trim();
    return res;}
friend Moly integ(Moly a) {
    int n = a.size();
    static vector<mint> inv = {mint::init(0), mint::init(1)};
    for (int i = inv.size(); i < n; i++) {
        inv.push_back(mint::init(NTT_P - NTT_P / i) * inv[NTT_P %
            i]);}
    a.v.resize(n + 1);
    for (int i = n; i >= 1; i--) { a[i] = a[i - 1] * inv[i]; }
    a[0] = mint::init(0); a.trim();
    return a;}
friend Moly invS(const Moly &a, int deg) { //  $A^{-1}(x) \bmod x^{\deg + 1}$ 
    assert(a[0] != mint::init(0));
    Moly res({inv(a[0])});
    for (int len = 2; len < 2 * deg + 2; len <= 1) {
        static vector<mint> fa, fb;
        fa.assign(a.v.begin(), a.v.begin() + min(int(a.size()),
            len));
        fa.resize(2 * len, mint::init(0));
        fb.assign(res.v.begin(), res.v.end());
        fb.resize(2 * len, mint::init(0));
        ntt(fa, false); ntt(fb, false);
        for (int i = 0; i < 2 * len; i++) { fb[i] = fb[i] *
            (mint::init(2) - fa[i] * fb[i]); }
        ntt(fb, true);
        res.v.assign(fb.begin(), fb.begin() + len);}
    res.v.resize(deg + 1, mint::init(0)); res.trim();
    return res;}
friend Moly log(const Moly &a, int deg) { //  $\ln(A(x)) = \text{integ}$ 
 $A'(x)/A(x) \bmod x^{\deg + 1}$ 
    assert(a.size() > 0 && a[0] == mint::init(1));
    Moly da = deriv(a);
    if (da.size() > deg) { da.v.resize(deg); }
    Moly res = integ(std::move(da) * invS(a, deg - 1));
    res.v.resize(deg + 1, mint::init(0)); res.trim();
    return res;}
friend Moly powS(const Moly &a, lng b, int deg) { //  $A(x)^b \bmod$ 
 $x^{\deg + 1}$ 
    assert(b >= 0);
    if (b == 0) { return Moly({mint::init(1)}); }
    if (a.isNil()) { return Moly({mint::init(0)}); }
    int n = a.size(), i = 0;
    while (i < n && a[i].n == 0) { i++; }
    if (i == n || (i > 0 && b > deg / i)) { return
        Moly({mint::init(0)}); }
    mint a_inv = inv(a[i]); Moly res(n - i);
    for (int j = i; j < n; j++) { res[j - i] = a[j] * a_inv; }
    int shf = b * i; mint a_pow = pow(a[i], b);

```

```

res = log(res, deg - shf); res *= mint(b);
res = exp(res, deg - shf); res.v.resize(deg + 1);
for (int j = deg; j >= shf; j--) { res[j] = res[j - shf] *
a_pow; }
for (int j = 0; j < shf; j++) { res[j] = mint::init(0); }
res.trim();
return res; }
// T: O(n * log(n)^2)
friend mint solveKthTerm(Moly a, Moly b, lng k) { // Bostan-Mori
assert(!b.isNil() && b[0].n != 0);
Moly b_neg;
while (k > 0) {
b_neg.v.assign(b.v.begin(), b.v.end());
for (int i = 1; i < b_neg.size(); i += 2) { b_neg[i] =
-b_neg[i]; }
a *= b_neg; b *= b_neg;
int a_len = 0;
for (int i = k & 1; i < a.size(); i += 2) { a[a_len++] =
a[i]; }
a.v.resize(a_len); a.trim();
int b_len = 0;
for (int i = 0; i < b.size(); i += 2) { b[b_len++] = b[i]; }
b.v.resize(b_len); b.trim();
k >= 1; }
return a.isNil() ? mint::init(0) : a[0] * inv(b[0]); }
friend Moly sqrt(const Moly &a, int deg) { // A(x)^{1/2} mod x^{deg}
+ 1 }
if (a.isNil()) { return Moly({mint::init(0)}); }
int n = a.size(), i = 0;
while (i < n && a[i].n == 0) { i++; }
if (i == n || i / 2 > deg) { return Moly({mint::init(0)}); }
if (i & 1) { return Moly({mint::init(-1)}); }
mint a_rt2 = sqrt(a[i]);
if (a_rt2.n == -1) { return Moly({mint::init(-1)}); }
int shf = i / 2; mint inv2 = inv(mint::init(2)); Moly
res({a_rt2});
for (int len = 2; len / 2 <= deg - shf; len <= 1) {
Moly cur(min(n - i, len));
for (int j = 0; j < cur.size(); j++) { cur[j] = a[i + j]; }
cur = std::move(cur) * invS(res, len - 1);
res.v.resize(len, mint::init(0));
for (int j = 0; j < cur.size(); j++) { res[j] = (res[j] +
cur[j]) * inv2; }
for (int j = cur.size(); j < len; j++) { res[j] *= inv2; }
res.trim(); }
res.v.resize(deg + 1, mint::init(0));
for (int j = deg; j >= shf; j--) { res[j] = res[j - shf]; }
for (int j = 0; j < shf; j++) { res[j] = mint::init(0); }
res.trim();
return res; }
// T: O(n^2)
static Moly calcMinLinRec(const vector<mint> &seq) { //
Berlekamp-Massey
if (seq.empty()) { return Moly({mint::init(1)}); }
int len = 0, p = -1; mint d_prv = mint::init(1);
Moly res({mint::init(1)}), cur({mint::init(1)}), tmp;
for (int i = 0; i < seq.size(); i++) {
mint d = mint::init(0);
for (int j = 0; j < min(len + 1, int(res.size())); j++) { d
+= res[j] * seq[i - j]; }
if (d.n == 0) { continue; }
int shf = i - p; mint dd = d * inv(d_prv);
if (2 * len <= i) { tmp = res; }
if (res.size() < cur.size() + shf) {
res.v.resize(cur.size() + shf, mint::init(0)); }
for (int j = 0; j < cur.size(); j++) { res[j + shf] -= dd *
cur[j]; }
if (2 * len <= i) { len = i - len + 1; p = i; d_prv = d;
std::swap(cur, tmp); } }
res.trim();
return res; }
// T: O(n^2)

```

```

static mint guessKthTerm(const vector<mint> &seq, lng k) {
if (k < seq.size()) { return seq[k]; }
Moly b = calcMinLinRec(seq);
if (b.size() <= 1) { return mint::init(0); }
Moly a(vector<mint>(seq.begin(), seq.begin() + b.size() - 1));
a = std::move(a) * b; a.v.resize(b.size() - 1); a.trim();
return solveKthTerm(std::move(a), std::move(b), k); }
// T: O(1)
bool isNil() const { return v.empty() || (v.size() == 1 && v[0].n
== 0); }
bool operator==(const Moly &o) const { return v == o.v; }
bool operator!=(const Moly &o) const { return !(*this == o); }
Moly &operator<=(int shf) {
if (size() == 0 || (size() == 1 && v[0] == mint::init(0))) {
return *this; }
v.insert(v.begin(), shf, mint::init(0));
return *this; }
Moly &operator>=(int shf) {
if (shf >= size()) { v = {mint::init(0)}; return *this; }
v.erase(v.begin(), v.begin() + shf);
return *this; }
friend Moly operator<<(Moly a, int shf) { a <= shf; return a; }
friend Moly operator>>(Moly a, int shf) { a >= shf; return a; }
friend ostream &operator<<(ostream &os, const Moly &a) {
if (a.size() == 0) { return os << "[]"; }
os << "[";
for (int i = 0; i < a.size(); i++) { os << a[i].n << (i <
a.size() - 1 ? ", " : ""); }
return os << "];"; }
};

```

5.28 Fast Is Square

```

#pragma once
#include "../1-Core/01-template.hpp"

```

```

// T: O(1), M: O(1)
bool isSquare(lng x) {
static constexpr ulng msk = []() {
ulng res = 0;
for (int i = 0; i < 64; i++) { res |= (1ULL << (63 - (i * i) %
64)); }
return res; }();
if (x < 0) { return false; }
if (x == 0) { return true; }
if ((lng(msk << (x & 63))) >= 0) { return false; }
int cnt = __builtin_ctzll(x);
if (cnt & 1) { return false; }
x >= cnt;
if ((x & 7) != 1) { return false; }
lng xrt = std::round(std::sqrtl(x));
return xrt * xrt == x; }

```

5.29 Josephus Problem

```

#pragma once
#include "../1-Core/01-template.hpp"

```

```

// T: O(n * log(n)), M: O(log(n))
lng josephus(lng n, lng k) {
if (n == 1) { return 0; }
if (k == 1) { return n - 1; }
if (n < k) {
lng res = 0;
for (lng i = 2; i <= n; i++) { res = (res + k) % i; }
return res; }
lng nk = n - n / k;
return k * ((josephus(nk, k) + nk - n % k % nk) % nk) / (k - 1); }

```

5.30 Simpson Integration

```

#pragma once
#include "../1-Core/01-template.hpp"

```

```

// T: O(n), M: O(1)
template<typename F>
double fixedIntegS(double a, double b, F f, int n = 1000000) {
n += n & 1; double res = f(a) + f(b), d = (b - a) / n;
for (int i = 1; i < n; i++) { res += f(a + i * d) * (2.0 + (i & 1)
* 2.0); }
return res * d / 3.0; }
// T: O(n), M: O(log(n))
template<typename F>
double adaptiveIntegS(double a, double b, F f, double eps = 1e-9) {
auto calc = [&](auto &&calc, double l, double r,
double fl, double fm, double fr, double res, double
eps) -> double {
double m = l + (r - l) / 2.0;
double flm = f(l + (m - l) / 2.0), lres = (fl + 4.0 * flm + fm)
* (m - l) / 6.0;
double frm = f(m + (r - m) / 2.0), rres = (fm + 4.0 * frm + fr)
* (r - m) / 6.0;
if (abs(lres + rres - res) <= 15.0 * eps) {
return lres + rres + (lres + rres - res) / 15.0; }
return calc(calc, l, m, fl, flm, fm, lres, eps / 2.0) +
calc(calc, m, r, fm, frm, fr, rres, eps / 2.0); }
double m = a + (b - a) / 2.0, fa = f(a), fm = f(m), fb = f(b);
return calc(calc, a, b, fa, fm, fb, (fa + 4.0 * fm + fb) * (b - a)
/ 6.0, eps); }

```

5.31 Solve Diophantine Equation

```

#pragma once
#include "../1-Core/01-template.hpp"
#include "../5-Mathematics/02-exgcd.hpp"

```

```

// T: O(log(n)), M: O(1)
bool solveDioEq(lng a, lng b, lng c, lng &x, lng &y, lng &g) { // ax +
by = c
if (a == 0 && b == 0) {
if (c != 0) { return false; }
x = y = g = 0;
return true; }
if (a == 0) {
if (c % b != 0) { return false; }
x = 0; y = c / b; g = abs(b);
return true; }
if (b == 0) {
if (c % a != 0) { return false; }
x = c / a; y = 0; g = abs(a);
return true; }
g = exGcd(abs(a), abs(b), x, y);
if (c % g != 0) { return false; }
if (a < 0) { x = -x; }
if (b < 0) { y = -y; }
x = lll(x) * (c / g) % abs(b / g);
if (x < 0) { x += abs(b / g); }
y = (c - lll(a) * x) / b;
return true; }

```

5.32 Solve Modulo Equation

```

#pragma once
#include "../1-Core/01-template.hpp"
#include "../5-Mathematics/02-exgcd.hpp"

```

```

// T: O(log(n)), M: O(1)
pair<lng, lng> solveModEq(lng a, lng b, lng m0) { // ax = b (mod m0)
a = (a % m0 + m0) % m0; b = (b % m0 + m0) % m0;

```

```
using safe_unordered_map = unordered_map<K, V, CustomHash>;
template<typename T>
using safe_unordered_set = unordered_set<T, CustomHash>;
```

6.1 Custom Hash

```

// S: NA, U:  $O(n \log n)$ , Q: NA, M:  $O(1)$ 
struct FastConv {
    template<typename T> static void fctOr(vector<T> &a, bool inv) {
        int n = int(a.size());
        for (int len = 1; len < n; len <= 1) {
            for (int i = 0; i < n; i += 2 * len) {
                for (int j = 0; j < len; j++) {
                    if (!inv) { a[i + j + len] += a[i + j]; }
                    else { a[i + j + len] -= a[i + j]; }
                }
            }
        }
    }

    template<typename T> static void fctAnd(vector<T> &a, bool inv) {
        int n = int(a.size());
        for (int len = 1; len < n; len <= 1) {
            for (int i = 0; i < n; i += 2 * len) {
                for (int j = 0; j < len; j++) {
                    if (!inv) { a[i + j + len] = a[i + j + len]; }
                    else { a[i + j + len] = a[i + j + len]; }
                }
            }
        }
    }

    template<typename T> static void fctXor(vector<T> &a, bool inv) {
        int n = int(a.size());
        for (int len = 1; len < n; len <= 1) {
            for (int i = 0; i < n; i += 2 * len) {
                for (int j = 0; j < len; j++) {
                    T u = a[i + j], v = a[i + j + len];
                    a[i + j] = u + v; a[i + j + len] = u - v;
                }
            }
            if (inv) {
                T ci = T(1) / T(n);
                for (auto &x : a) { x *= ci; }
            }
        }
    }

    template<typename T> static void fctGcdSlow(vector<T> &a, bool inv) {
        int n = int(a.size()) - 1;
        if (!inv) {
            for (int i = n; i >= 1; i--) {
                for (int j = 2 * i; j <= n; j += i) { a[i] += a[j]; }
            }
        } else {
            for (int i = 1; i <= n; i++) {
                for (int j = 2 * i; j <= n; j += i) { a[i] -= a[j]; }
            }
        }
    }

    template<typename T> static void fctGcd(vector<T> &a, const
vector<int> &prms, bool inv) {
        int n = int(a.size()) - 1;
        if (!inv) {
            for (int p : prms) {
                if (p > n) break;
                for (int i = n / p; i >= 1; i--) { a[i] += a[i * p]; }
            }
        } else {
            for (int p : prms) {
                if (p > n) break;
                for (int i = 1; i <= n / p; i++) { a[i] -= a[i * p]; }
            }
        }
    }

    template<typename T> static void fctLcmSlow(vector<T> &a, bool inv) {
        int n = int(a.size()) - 1;
        if (!inv) {

```

6.3 Kadane's Algorithm

```

template<typename T>
lng kadane(const vector<T> &arr) {
    if (arr.empty()) {
        return 0;
    }

    lng max_sum = -INF64, cur_sum = -INF64;
    for (T a : arr) {
        cur_sum = max(cur_sum + a, lng(a));
        max_sum = max(max_sum, cur_sum);
    }

    return max_sum;
}

```

6.4 Longest Common Subsequence

```

def lcs(X, Y, m, n):
    L = [[0 for i in range(n+1)] for j in range(m+1)]
    for i in range(m+1):
        for j in range(n+1):
            if i == 0 or j == 0:
                L[i][j] = 0
            elif X[i-1] == Y[j-1]:
                L[i][j] = L[i-1][j-1] + 1
            else:
                L[i][j] = max(L[i-1][j], L[i][j-1])

    lcs = ""
    i = m
    j = n
    while i > 0 and j > 0:
        if X[i-1] == Y[j-1]:
            lcs += X[i-1]
            i -= 1
            j -= 1
        elif L[i-1][j] > L[i][j-1]:
            i -= 1
        else:
            j -= 1
    lcs = lcs[::-1]
    print(lcs)

#test
X = "AGGTAB"
Y = "GXTXAYB"
m = len(X)
n = len(Y)
lcs(X, Y, m, n)

```

6.4 Longest Common Subsequence

```
def lcs(X, Y, m, n):
    L = [[0 for i in range(n+1)] for j in range(m+1)]
    for i in range(m+1):
        for j in range(n+1):
            if i == 0 or j == 0:
                L[i][j] = 0
            elif X[i-1] == Y[j-1]:
                L[i][j] = L[i-1][j-1] + 1
            else:
                L[i][j] = max(L[i-1][j], L[i][j-1])

    lcs = ""
    i = m
    j = n
    while i > 0 and j > 0:
        if X[i-1] == Y[j-1]:
            lcs += X[i-1]
            i -= 1
            j -= 1
        elif L[i-1][j] > L[i][j-1]:
            i -= 1
        else:
            j -= 1
    lcs = lcs[::-1]
    print(lcs)

#test
X = "AGGTAB"
Y = "GXTXAYB"
m = len(X)
n = len(Y)
lcs(X, Y, m, n)
```

$$\left(x + k \frac{b}{\gcd(a, b)}, y - k \frac{a}{\gcd(a, b)}\right)$$

7.5 Misc

7.5.1 Determinants and PM

$$\begin{aligned}
 \det(A) &= \sum_{\sigma \in S_n} \text{sgn}(\sigma) \prod_{i=1}^n a_{i, \sigma(i)} \\
 \text{perm}(A) &= \sum_{\sigma \in S_n} \prod_{i=1}^n a_{i, \sigma(i)} \\
 pf(A) &= \frac{1}{2^n n!} \sum_{\sigma \in S_{2n}} \text{sgn}(\sigma) \prod_{i=1}^n a_{\sigma(2i-1), \sigma(2i)} \\
 &= \sum_{M \in \text{PM}(n)} \text{sgn}(M) \prod_{(i,j) \in M} a_{i,j}
 \end{aligned}$$

7.5.2 BEST Theorem

Count directed Eulerian cycles. Number of OST given by Kirchoff's Theorem (remove r/c with root) $\# \text{OST}(G, r) \cdot \prod_v (d_v - 1)!$

7.5.3 Primitive Roots

Only exists when n is $2, 4, p^k, 2p^k$, where p odd prime. Assume n prime. Number of primitive roots $\phi(\phi(n))$. Let g be primitive root. All primitive roots are of the form g^k where $k, \phi(p)$ are coprime.
 k -roots: $g^{i \cdot \phi(n)/k}$ for $0 \leq i < k$

7.5.4 Sum of primes

For any multiplicative f :

$$S(n, p) = S(n, p-1) - f(p) \cdot (S(n/p, p-1) - S(p-1, p-1))$$

7.5.5 Floor

$$\begin{aligned}
 \lfloor \lfloor x/y \rfloor / z \rfloor &= \lfloor x / (yz) \rfloor \\
 x \% y &= x - y \lfloor x/y \rfloor
 \end{aligned}$$

7.5.6 Large Primes

100894373, 103893941, 999999937, 1000000007, 16208191877

7.6 Other Combinatorics Stuff

Catalan	$C_0 = 1, C_n = \frac{1}{n+1} \binom{2n}{n} = \sum_{i=0}^{n-1} C_i C_{n-i-1} = \frac{4n-2}{n+1} C_{n-1}$	
Stirling 1st kind	$\begin{bmatrix} 0 \\ 0 \end{bmatrix} = 1, \begin{bmatrix} n \\ 0 \end{bmatrix} = \begin{bmatrix} 0 \\ n \end{bmatrix} = 0, \begin{bmatrix} n \\ k \end{bmatrix} = (n-1) \begin{bmatrix} n-1 \\ k \end{bmatrix} + \begin{bmatrix} n-1 \\ k-1 \end{bmatrix}$	#perms of n objs with exactly k cycles
Stirling 2nd kind	$\left\{ \begin{smallmatrix} n \\ 1 \end{smallmatrix} \right\} = \left\{ \begin{smallmatrix} n \\ n \end{smallmatrix} \right\} = 1, \left\{ \begin{smallmatrix} n \\ k \end{smallmatrix} \right\} = k \left\{ \begin{smallmatrix} n-1 \\ k \end{smallmatrix} \right\} + \left\{ \begin{smallmatrix} n-1 \\ k-1 \end{smallmatrix} \right\}$	#ways to partition n objs into k nonempty sets
Euler	$\langle \begin{smallmatrix} n \\ 0 \end{smallmatrix} \rangle = \langle \begin{smallmatrix} n \\ n-1 \end{smallmatrix} \rangle = 1, \langle \begin{smallmatrix} n \\ k \end{smallmatrix} \rangle = (k+1) \langle \begin{smallmatrix} n-1 \\ k \end{smallmatrix} \rangle + (n-k) \langle \begin{smallmatrix} n-1 \\ k-1 \end{smallmatrix} \rangle$	#perms of n objs with exactly k ascents
Euler 2nd Order	$\ll \begin{smallmatrix} n \\ k \end{smallmatrix} \gg = (k+1) \ll \begin{smallmatrix} n-1 \\ k \end{smallmatrix} \gg + (2n-k-1) \ll \begin{smallmatrix} n-1 \\ k-1 \end{smallmatrix} \gg$	#perms of $1, 1, 2, 2, \dots, n, n$ with exactly k ascents
Bell	$B_1 = 1, B_n = \sum_{k=0}^{n-1} B_k \binom{n-1}{k} = \sum_{k=0}^n \left\{ \begin{smallmatrix} n \\ k \end{smallmatrix} \right\}$	#partitions of $1..n$ (Stirling 2nd, no limit on k)

#labeled rooted trees

$$n^{n-1}$$

#labeled unrooted trees

$$n^{n-2}$$

#forests of k rooted trees

$$\frac{k}{n} \binom{n}{k} n^{n-k}$$

$$\sum_{i=1}^n i^2 = n(n+1)(2n+1)/6$$

$$\sum_{i=1}^n i^3 = n^2(n+1)^2/4$$

$$!n = n \times! (n-1) + (-1)^n$$

$$!n = (n-1)!(n-1) +!(n-2))$$

$$\sum_{i=1}^n \binom{n}{i} F_i = F_{2n}$$

$$\sum_i \binom{n-i}{i} = F_{n+1}$$

$$\sum_{k=0}^n \binom{k}{m} = \binom{n+1}{m+1}$$

$$x^k = \sum_{i=0}^k i! \left\{ \begin{smallmatrix} k \\ i \end{smallmatrix} \right\} \binom{x}{i} = \sum_{i=0}^k \left\langle \begin{smallmatrix} k \\ i \end{smallmatrix} \right\rangle \binom{x+i}{k}$$

$$a \equiv b \pmod{x, y} \Rightarrow a \equiv b \pmod{\text{lcm}(x, y)}$$

$$\sum_{d|n} \phi(d) = n$$

$$ac \equiv bc \pmod{m} \Rightarrow a \equiv b \pmod{\frac{m}{\text{gcd}(c, m)}}$$

$$(\sum_{d|n} \sigma_0(d))^2 = \sum_{d|n} \sigma_0(d)^3$$

$$p \text{ prime} \Leftrightarrow (p-1)! \equiv -1 \pmod{p}$$

$$\text{gcd}(n^a - 1, n^b - 1) = n^{\text{gcd}(a, b)} - 1$$

$$\sigma_x(n) = \prod_{i=0}^r \frac{p_i^{(a_i+1)x} - 1}{p_i^x - 1}$$

$$\sigma_0(n) = \prod_{i=0}^r (a_i + 1)$$

$$\sum_{k=0}^m (-1)^k \binom{n}{k} = (-1)^m \binom{n-1}{m}$$

$$2^{\omega(n)} = O(\sqrt{n})$$

$$\sum_{i=1}^n 2^{\omega(i)} = O(n \log n)$$

$$d = v_i t + \frac{1}{2} a t^2$$

$$v_f^2 = v_i^2 + 2ad$$

$$v_f = v_i + at$$

$$d = \frac{v_i + v_f}{2} t$$

7.7 The Twelfold Way

Putting n balls into k boxes.

	Balls Boxes	same same	distinct same	same distinct	distinct distinct	Remarks
-	-	$p_k(n)$	$\sum_{i=0}^k \left\{ \begin{smallmatrix} n \\ i \end{smallmatrix} \right\}$	$\binom{n+k-1}{k-1}$	k^n	$p_k(n)$: #partitions of n into $\leq k$ positive parts
size ≥ 1	-	$p(n, k)$	$\left\{ \begin{smallmatrix} n \\ k \end{smallmatrix} \right\}$	$\binom{n-1}{k-1}$	$k! \left\{ \begin{smallmatrix} n \\ k \end{smallmatrix} \right\}$	$p(n, k)$: #partitions of n into k positive parts
size ≤ 1	-	$[n \leq k]$	$[n \leq k]$	$\binom{k}{n}$	$n! \binom{k}{n}$	$[cond]$: 1 if $cond = true$, else 0